

JOIG 2021 A問題 金平糖(Konpeito)

解説: 平木 康傑

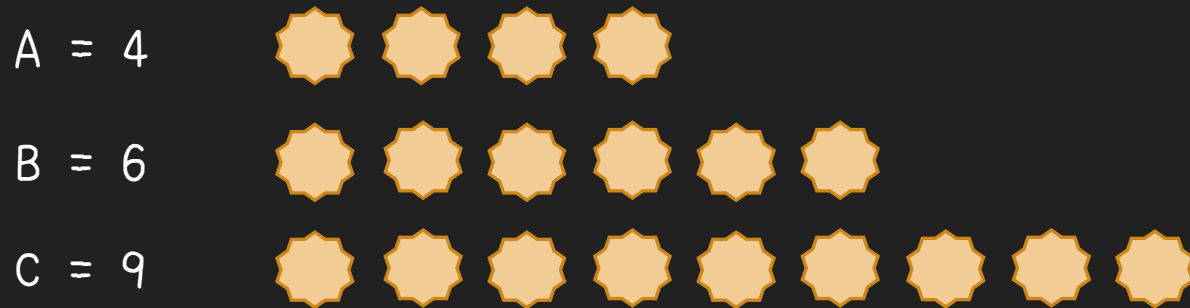


問題概要

- 3人が金平糖を同じ数だけ食べることが目標
- 現時点で3人が食べた個数が A, B, C として与えられる
- 追加で食べるべき個数の合計は最小でいくつ？

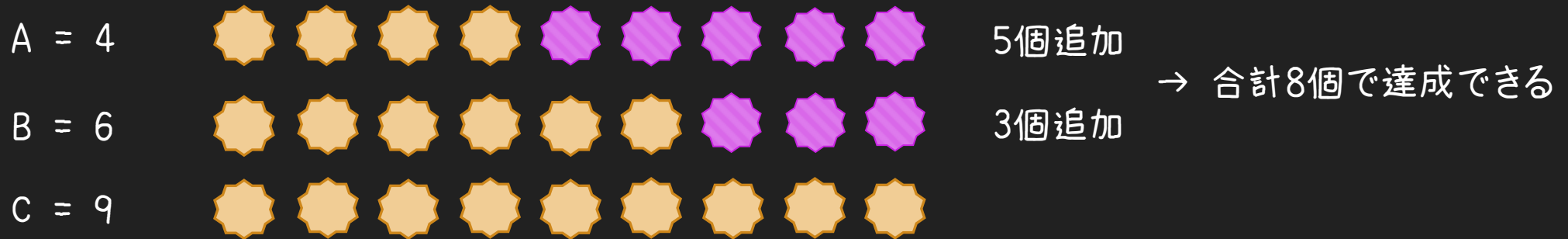
問題概要

- 3人が金平糖を同じ数だけ食べることが目標
- 現時点で3人が食べた個数が A , B , C として与えられる
- 追加で食べるべき個数の合計は最小でいくつ？



問題概要

- 3人が金平糖を同じ数だけ食べることが目標
- 現時点で3人が食べた個数が A, B, C として与えられる
- 追加で食べるべき個数の合計は最小でいくつ？



解法の方針

- A, B, C の値を受け取る(入力する)
- 答えを計算する
- 答えを出力する

解法の方針

- A, B, C の値を受け取る(入力する)
- 答えを計算する
- 答えを出力する
- 言語によって書き方は異なるので、入門書やチュートリアルなどを参考にしてある程度習得しておこう
 - 入力・出力
 - 四則演算など
 - 条件分岐

入力・出力

○ C++

- C++標準で用意された関数が用途ごとに別の「ライブラリ」に詰め込まれている
- 入力・出力関連のライブラリ関数は<iostream>ライブラリに入っている
 - #include <iostream> と書くことで取り込める
- C++の入出力は、標準入出力と結びついた「ストリーム」と演算子とを使って行う
 - std::cin は標準入力, std::cout は標準出力
 - 変数 a, b, c に入力: `std::cin >> a >> b >> c;`
 - 変数 result と改行を出力: `std::cout << result << '\n';`

入力・出力

○ Python

- Pythonでははじめから組み込み関数が見える

- 整数 a , b , c を入力: `a, b, c = map(int, input().split())`

 - `input()` で入力を文字列として受け取り, `.split()` で空白ごとに分割して配列にする

 - `map` 関数によって, その配列の各要素に `int` 関数を適用する

 - Python特有の書き方として, たとえば `a, b, c = [1, 2, 3]` とすればまとめて代入できる

- 変数 `result` を 1 行に出力: `print(result)`

解法の方針（再掲）

- ~~⊖ A, B, C の値を受け取る(入力する)~~ ✓
- 答えを計算する
- ~~⊖ 答えを出力する~~ ✓

考察

- 3人が金平糖を同じ数だけ食べることが目標
- いくつにすればいいか？

A = 4



B = 6



C = 9

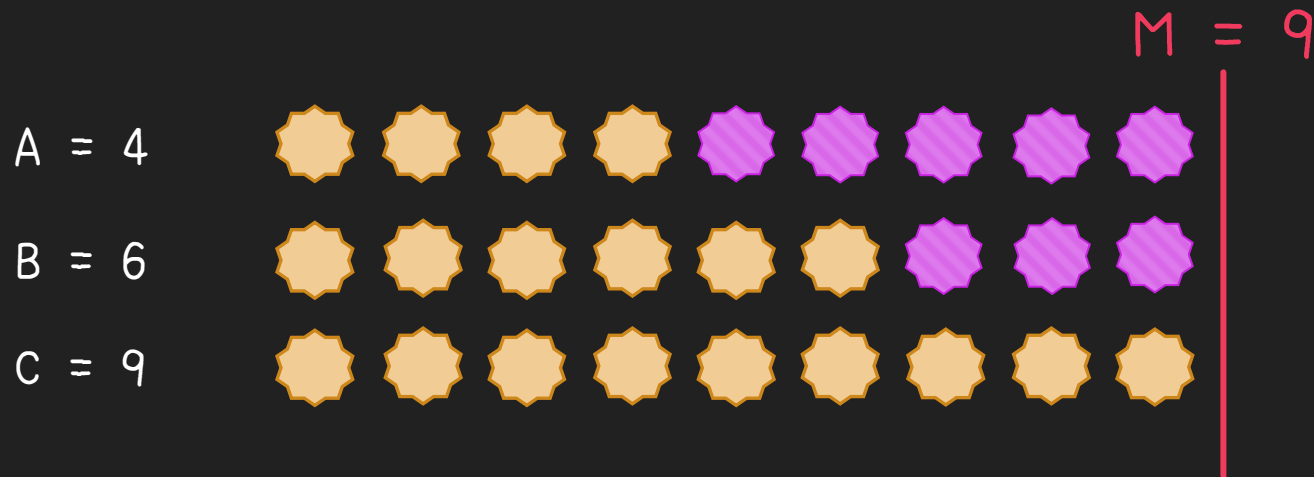


9



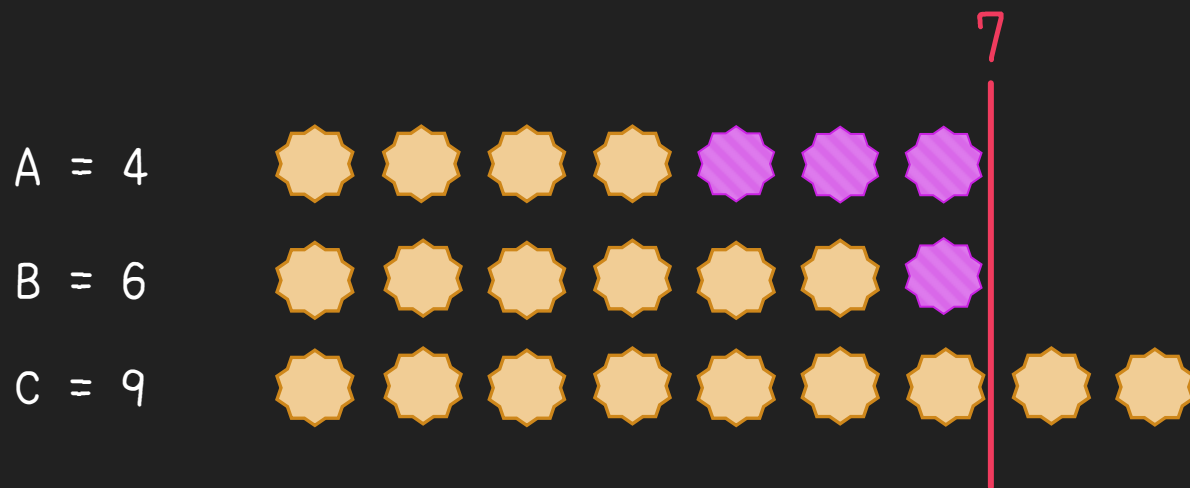
考察

- 3人が金平糖を同じ数だけ食べることが目標
- いくつにすればいいか？
- A, B, C の最大値 (M とする) にすればいい！
- なぜ？



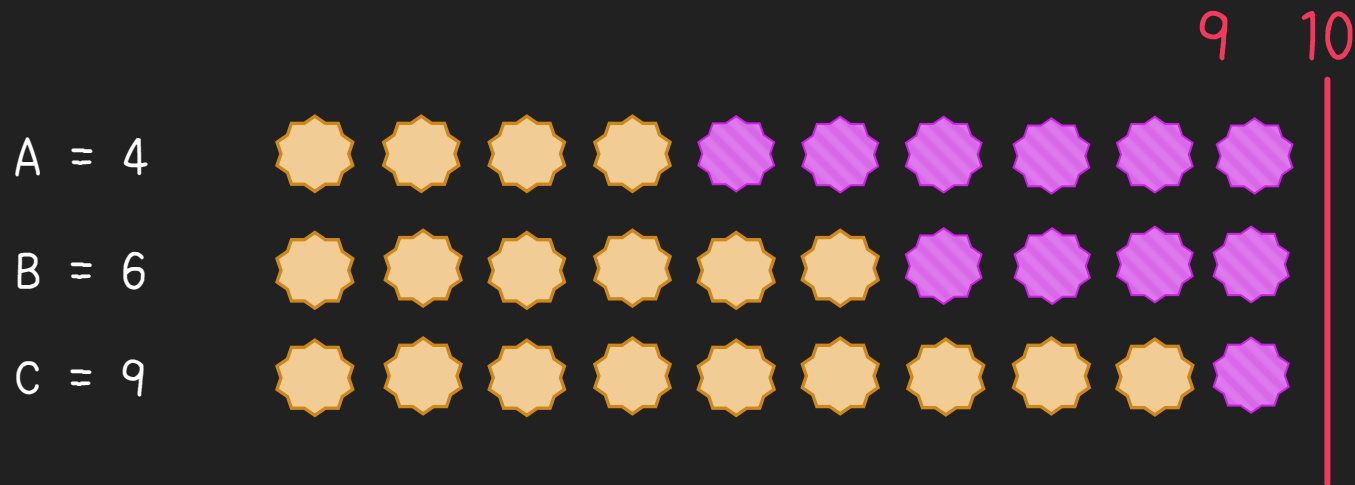
考察 (最大値 M にそろえるべき理由)

- M より小さい個数にそろえることは不可能
 - (もう M 個食べた人がいるから)



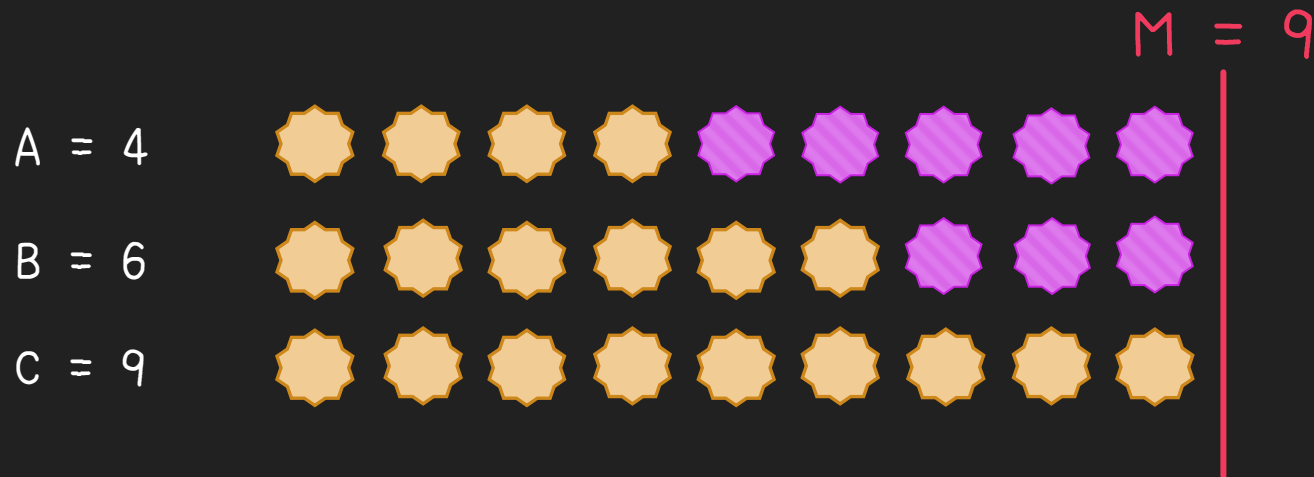
考察 (最大値 M にそろえるべき理由)

- M より大きい個数にそろえるのは無駄がある
 - 全員が追加で1つ以上食べているので、1ずつ減らして問題ない



考察

- なので、A, B, Cのうちの最大値を求めれば、それを M とおくと $(M-A) + (M-B) + (M-C)$ が答えとなる



最大値の求め方

- 小課題 1 では $A < B < C$ が保証されているので、いつでも C が最大値
 - 40点がもらえる
- 小課題 2 ではちゃんと最大値を求める必要がある

最大値の求め方

- 場合分け

- $(A \geq B \text{ かつ } A \geq C)$ なら A , $(B \geq C \text{ かつ } B \geq A)$ なら B , どちらでもなければ C

- 順に更新

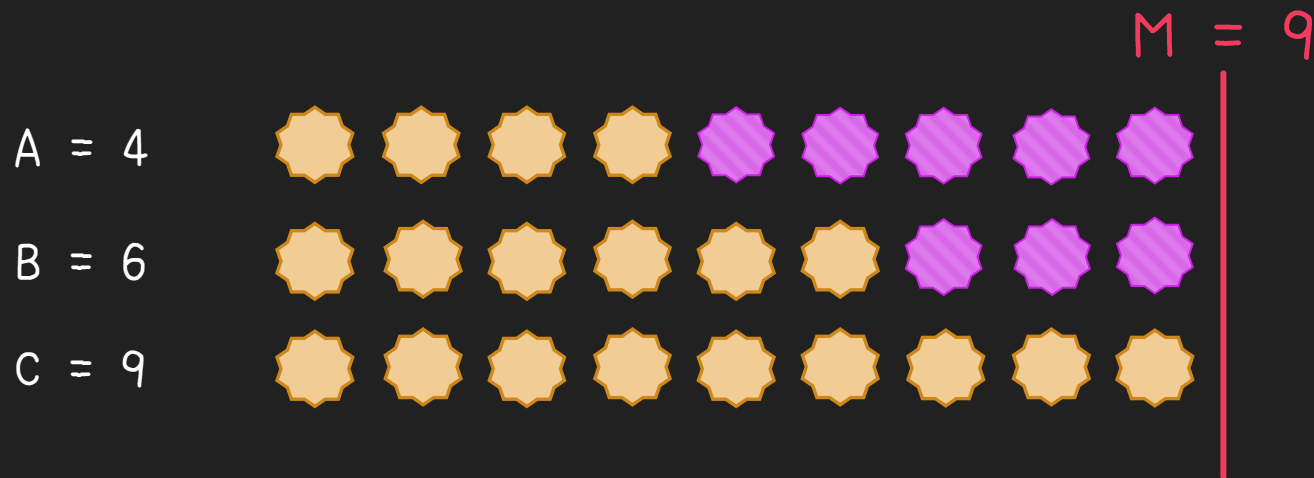
- はじめ $M = 0$ とする

- A, B, C と順番に比べ, M と比較対象のうち大きい方を新たな M にする

- 条件分岐を適切に使う

考察 (再掲)

- A, B, Cのうちの最大値を求めれば, それを M とおくと $(M-A) + (M-B) + (M-C)$ が答えとなる



解法の方針（再掲）

- ~~⊖ A, B, C の値を受け取る(入力する)~~ ✓
- ~~⊖ 答えを計算する~~ ✓
- ~~⊖ 答えを出力する~~ ✓

実装例

```
#include <iostream>

int main (void) {
    int A, B, C;

    std::cin >> A >> B >> C;

    int maxv = 0;
    if (maxv < A) maxv = A;
    if (maxv < B) maxv = B;
    if (maxv < C) maxv = C;

    int result = 0;
    result += (maxv - A);
    result += (maxv - B);
    result += (maxv - C);

    std::cout << result << '\n';

    return 0;
}
```

C++

```
import java.util.*;
```

```
public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner(System.in);

        int a = Integer.parseInt(sc.next());
        int b = Integer.parseInt(sc.next());
        int c = Integer.parseInt(sc.next());

        int maxv = 0;
        if (a >= b && a >= c) {
            maxv = a;
        } else if (b >= c && b >= a) {
            maxv = b;
        } else {
            maxv = c;
        }

        int result = (maxv - a) + (maxv - b) + (maxv - c);

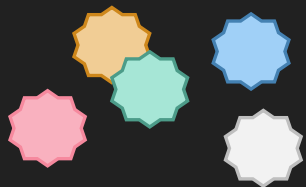
        System.out.println(result);
    }
}
```

Java

得点分布

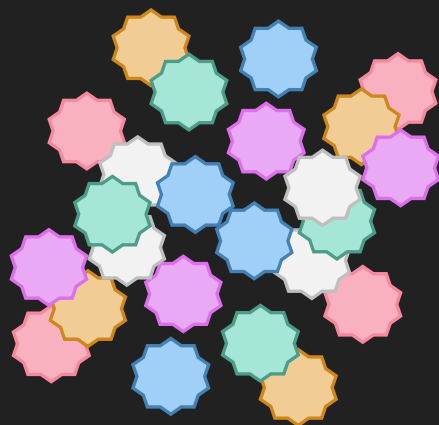
0点

5人



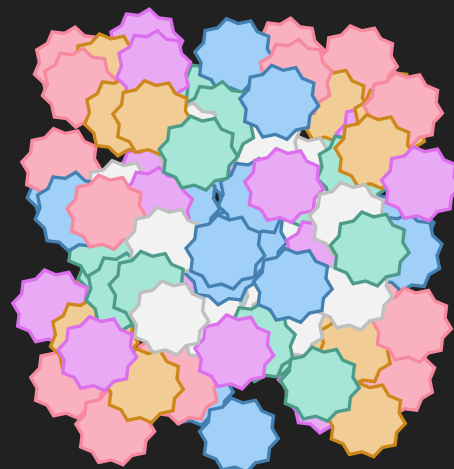
40点

24人



100点

96人



卷物 Scroll

JOIG 2021 第2問



問題概要

- 長さ N の文字列 S がある
- S の各文字は $joiJOI$ のいずれか
- S の K 番目の文字以降の大文字と小文字を逆転させたものが文字列 T
- 文字列 T が与えられるので元の文字列 S を求めよ



考察

文字列 S

JOIJOIJOIJOIJOI

K=10



文字列 T

JOIJOIJOI joiioi

どうやって
復元する？



考察



→二回操作すると元に戻る



考察

文字列 S

JOIJOIJOIJOIJOI



文字列 T

JOIJOIJOI joijoi

どうやって
復元する？

K 番目の文字以降の
大文字と小文字を反転させる



方針の一例

- for文などループを用いて1文字ずつ処理しましょう
- if文など条件分岐を用いて各文字の大文字と小文字を逆転させましょう



实装例 (C++)

```
#include <iostream>
#include <string>
using namespace std;

int main(){
    int N,K;
    string T;
    cin>>N>>K>>T;
    for(int i=0;i<N;i++){
        if(i<K-1){
            cout<<T[i];
        }
        else{
            if(T[i]=='j'){
                cout<<'J';
            }
            else if(T[i]=='o'){
                cout<<'0';
            }
            else if(T[i]=='i'){
                cout<<'I';
            }
            else if(T[i]=='J'){
                cout<<'j';
            }
            else if(T[i]=='0'){
                cout<<'o';
            }
            else if(T[i]=='I'){
                cout<<'i';
            }
        }
    }
    cout<<endl;
}
```

```
#include <iostream>
#include <string>
using namespace std;

int main(){
    int N,K;
    string T;
    cin>>N>>K>>T;
    for(int i=K-1;i<N;i++){
        if('a'<=T[i]&&T[i]<='z'){
            T[i]=T[i]-'a'+'A';
        }
        else{
            T[i]=T[i]-'A'+'a';
        }
    }
    cout<<T<<endl;
}
```



実装例 (Python)

```
N, K = map(int, input().split())  
T = input()  
print(T[:K-1] + T[K-1:].swapcase())
```

この実装例は「方針の一例」の方針とは異なります



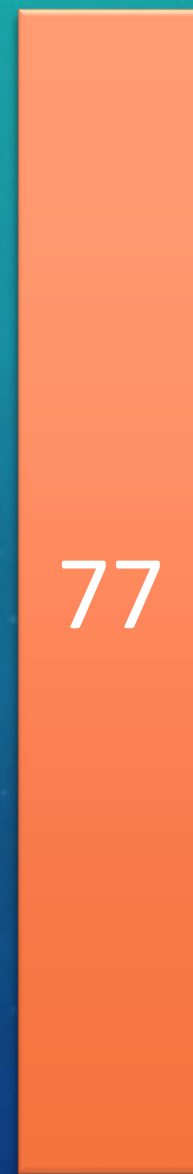
得点分布



0



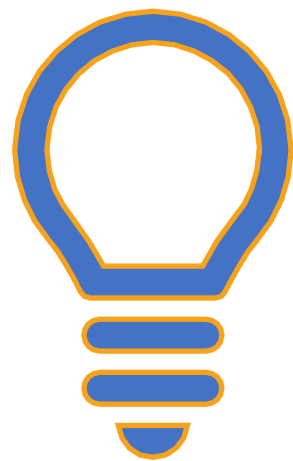
30



100

(人)

(点)



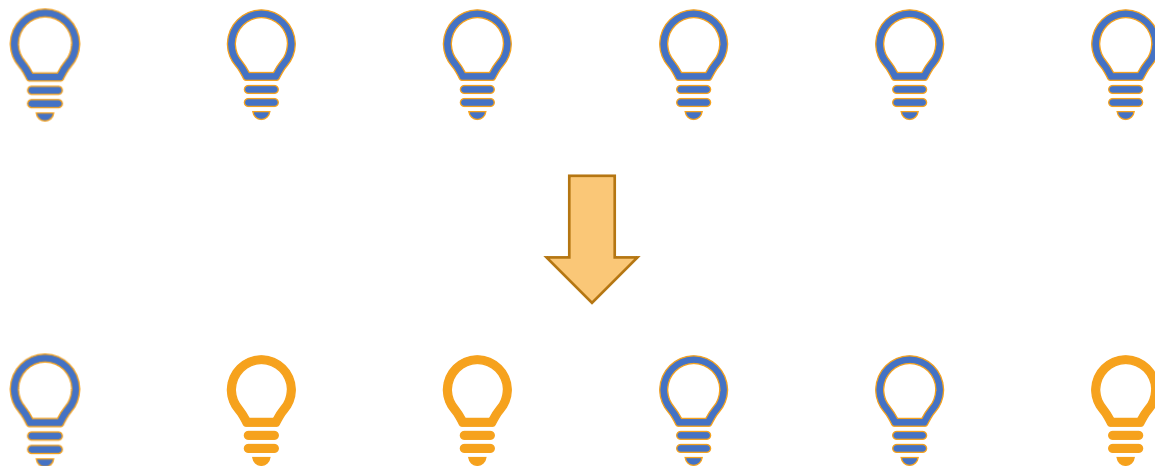
JOIG 2021 第3問 イルミネーション 2 (Illumination 2)

解説 nxteru

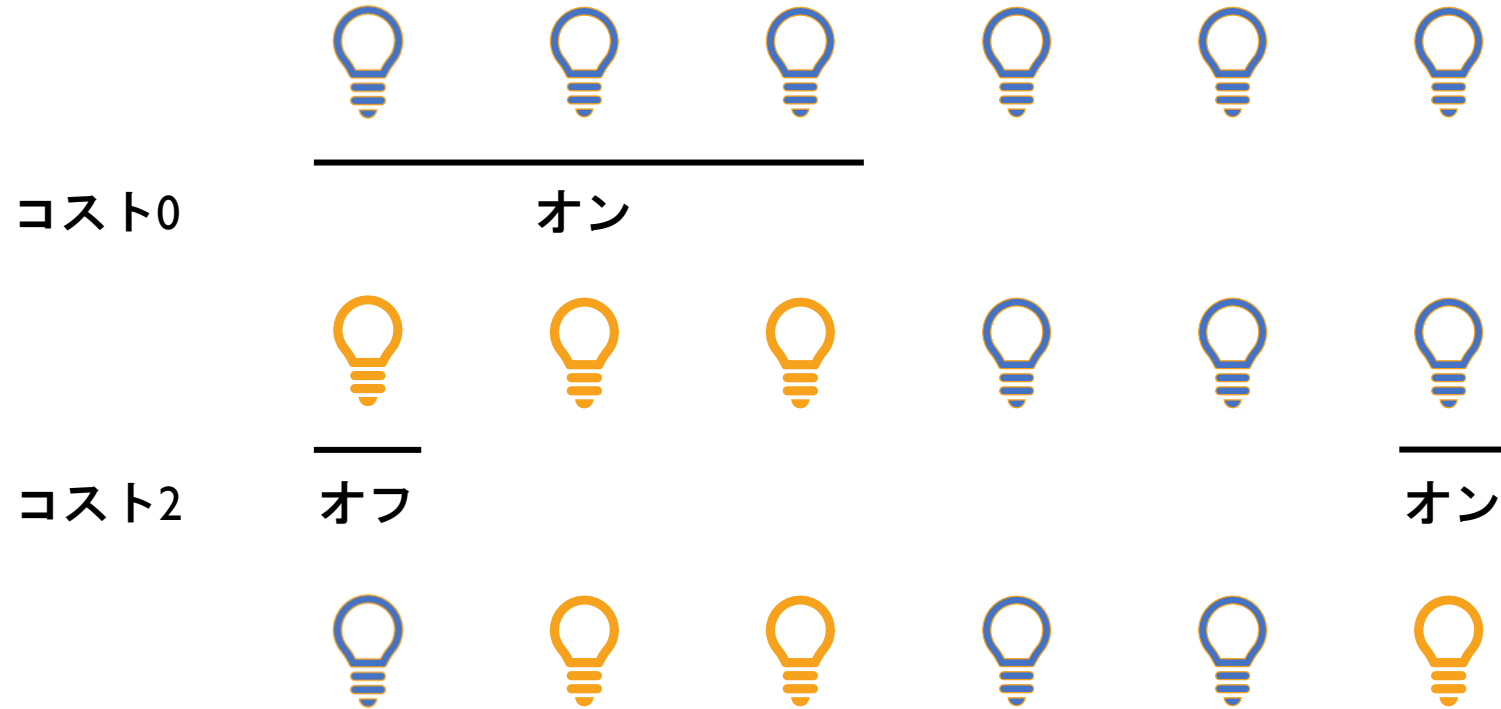
問題概要

- N 個の電球があり最初はすべてオフ
 - 最初に無料で電球 $1, 2, \dots, r$ をすべてオンにすることができる
 - その後コスト1につき好きな電球1つのオン／オフを切り替えられる
 - 電球の状態を $A_1, A_2, \dots, A_N$ にするためにかかるコストの最小値は？
-
- 小課題
 1. $N \leq 2\,000$
 2. $N \leq 200\,000$

入力例 1



入力例1



考察

- オン／オフを切り替える操作について
- 最初の操作が終わった後の状態を $X_1, X_2, \dots, X_N$ とする
- X_i と A_i が同じなら電球 i は何もしない, 違うなら電球 i は切り替える
- 各電球について操作をするかしないかが定まる
- 最初の操作が決まれば, かかるコストも求まる

小課題 I

- 最初の操作を全探索する
- 最初の操作として考えられるものは
「なし」「1をオン」「1,2をオン」「1,2,3をオン」...「1,2,3,...,Nをオン」
の $N + 1$ 通り
- それぞれについてかかるコストを $O(N)$ で求める
- 計算量は $O(N^2)$

小課題2

- 各最初の操作に対するコスト計算を高速化する
- 最初の操作で「 $1, 2, \dots, r$ をオン」にしたときのコストは目標の模様から「 $A_1, A_2, \dots, A_r$ で0の個数」 + 「 $A_{r+1}, A_{r+2}, \dots, A_N$ で1の個数」で計算できる
- これは累積和を事前に計算しておくことで各 r について $O(1)$ でコストを計算できる

累積和

- 求めたいのは各 r に対する「 $A_1, A_2, \dots, A_r$ で0の個数」「 $A_{r+1}, A_{r+2}, \dots, A_N$ で1の個数」
- $S_i = A_1, A_2, \dots, A_i$ で0の個数と定義する
- $S_i = S_{i-1} + (A_iが0なら1, A_iが1なら0)$ と S_i の計算に S_{i-1} を利用できる
- i の小さい順に S_i を更新することで $O(N)$ で配列 S を計算できる
- 同様に $T_i = A_i, A_{i+1}, \dots, A_N$ で1の個数とする
- $T_i = T_{i+1} + A_i$ だから i の大きい順に更新することで $O(N)$ で配列 T を計算

まとめ

- 最初の操作で「 $1, 2, \dots, r$ をオン」にしたときのコストは $S_r + T_{r+1}$ となる
- 配列 S, T の前計算に $O(N)$,最初の操作の全探索で $O(N)$
- これで満点が獲得できる

得点分布（有資格者）

100点	22人
45点	9人
0点	94人

展覧会 2 [Exhibition 2]

日本情報オリンピック女性部門 (JOIG 2021)

2021.4.25

解説：E869120

1

問題概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

1



いくつかの絵が廊下に置かれている
各絵には価値が付けられている

2



N 個の絵のうち M 個を残し
その他をすべて取り外す

絵の間隔を全部 D 以上にせねばならない

1

問題概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

1

価値が最小となる絵の価値を

いくつかの絵が廊下に置かれている
各絵には価値が付けられている

最大化したい

2

N 個の絵のうち M 個を残し
その他をすべて取り外す

絵の間隔を全部 K 以上にせねばならない

1

具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

価値 1

価値 1

価値 2

価値 2

価値 2

価値 2



1

2

3

4

5

6

7

8

9

 $M = 3$ 個の絵を残すが距離は $D = 4$ 以上離す必要がある

1

具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

価値 1



価値 1



価値 2



価値 2



価値 2



価値 2



1

2

3

4

5

6

7

8

9

たとえばこんな感じの残し方を考える

1 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

価値 1

価値 1

価値 2

価値 2

価値 2

価値 2

この場合、価値の最小値は 1
それを 2 以上にする方法は存在しない
ため、答えは 1 である

たとえばこんな感じの残し方を考える

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	
2	$N = M$	
3	$N \leq 15$	
4	$N \leq 1000, V_i = 2$	
5	$N \leq 1000$	
6	$N \leq 10^5$	

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	
3	$N \leq 15$	
4	$N \leq 1000, V_i = 2$	
5	$N \leq 1000$	
6	$N \leq 10^5$	

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	絵を座標の小さい順にソートしてチェックする
3	$N \leq 15$	
4	$N \leq 1000, V_i = 2$	
5	$N \leq 1000$	
6	$N \leq 10^5$	

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	絵を座標の小さい順にソートしてチェックする
3	$N \leq 15$	bit 全探索をしよう
4	$N \leq 1000, V_i = 2$	
5	$N \leq 1000$	
6	$N \leq 10^5$	

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	絵を座標の小さい順にソートしてチェックする
3	$N \leq 15$	bit 全探索をしよう
4	$N \leq 1000, V_i = 2$	「答えは2か？」 → 「答えは1か？」
5	$N \leq 1000$	
6	$N \leq 10^5$	

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	絵を座標の小さい順にソートしてチェックする
3	$N \leq 15$	bit 全探索をしよう
4	$N \leq 1000, V_i = 2$	「答えは2か？」 → 「答えは1か？」
5	$N \leq 1000$	答えは $V_1, V_2, \dots, V_N$ のいずれかなので全探索する
6	$N \leq 10^5$	

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	絵を座標の小さい順にソートしてチェックする
3	$N \leq 15$	bit 全探索をしよう
4	$N \leq 1000, V_i = 2$	「答えは2か？」 → 「答えは1か？」
5	$N \leq 1000$	答えは $V_1, V_2, \dots, V_N$ のいずれかなので全探索する
6	$N \leq 10^5$	答えで二分探索

1 小課題ごとの解法概要

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

キーワードだけでは分からないと思うので
以降のスライドでは**小課題ごとに詳しく解説**します



目次

はじめに

1 ページ

小課題 1 ($M = 1$)、小課題 2 ($N = M$)

16 ページ

小課題 3 ($N \leq 15$)

35 ページ

小課題 4・5／満点解法

50 ページ

まとめ

80 ページ

2 小課題 1 | $M = 1$

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

1 個しか絵を残せないなので、**価値が最も大きい絵**を残すのが最適

2 小課題 1 | $M = 1$

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

1 個しか絵を残せないなので、**価値が最も大きい絵**を残すのが最適

答えは $\max(V_1, V_2, \dots, V_N)$ となり、実装例は次の通り

実装例 (C++)

```
#include <iostream>
using namespace std;
int N, M, D, X[200009], V[200009];

int main() {
    cin >> N >> M >> D;
    for (int i = 1; i <= N; i++) cin >> X[i] >> V[i];
    for (int i = 1; i <= N; i++) Answer = max(Answer, V[i]);
    cout << Answer << endl;
    return 0;
}
```

3点

2 小課題 2 | $N = M$

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- すべての絵を残せるので、この残し方が条件を満たすか、つまり距離が D 未満となっている絵の組が存在するか判定すれば良い

2 小課題 2 | $N = M$

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● すべての絵を残せるので、この残し方が条件を満たすか、つまり距離が D 未満となっている絵の組が存在するか判定すれば良い

全部距離 D 以上の場合 → 答えは $\min(V_1, V_2, \dots, V_N)$

2 小課題 2 | $N = M$

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● すべての絵を残せるので、この残し方が条件を満たすか、つまり距離が D 未満となっている絵の組が存在するか判定すれば良い

全部距離 D 以上の場合 → 答えは $\min(V_1, V_2, \dots, V_N)$

そうでない場合 → 答えは -1

2 小課題 2 | $N = M$

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● すべての絵を残せるので、この残し方が条件を満たすか、つまり距離が K 未満となっている絵の組が存在するか判定すれば良い

どうやって判定するか？
全部距離 K 以上の場合 → 答えは $\min(V_1, V_2, \dots, V_N)$

そうでない場合 → 答えは -1

2 小課題 2 | $N = M$ (低速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 最も単純な方法として、以下のように全部の絵の組に対して距離を計算するアルゴリズムが考えられる

実装例 (C++)

```
bool check() {  
    for (int i = 1; i <= N; i++) {  
        for (int j = i + 1; j <= N; j++) {  
            int dist = abs(X[i] - X[j]);  
            if (dist < D) return false;  
        }  
    }  
    return true;  
}
```

2 小課題 2 | $N = M$ (低速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

最も単純な方法として、以下のように全部の絵の組に対して距離を計算するアルゴリズムが考えられる

実装例 (C++)

```
bool check() {  
    for (int i = 1; i <= N; i++) {  
        for (int j = i + 1; j <= N; j++) {  
            int dist = abs(X[i] - X[j]);  
            if (dist < D) return false;  
        }  
    }  
    return true;  
}
```

しかしこれだと TLE する

2 小課題 2 | $N = M$ (低速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 一般に、競技プログラミングにおいて、計算回数は $10^8 \sim 10^9$ 回程度までしか許されない

2 小課題 2 | $N = M$ (低速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 一般に、競技プログラミングにおいて、計算回数は $10^8 \sim 10^9$ 回程度までしか許されない
 - $N = 10^5$ で計算量 $O(N^2)$ の二重ループを書くと、計算回数が 10^{10} 程度となり、数秒以内に実行が終わらず TLE となってしまう

2 小課題 2 | $N = M$ (低速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 一般に、競技プログラミングにおいて、計算回数は $10^8 \sim 10^9$ 回程度までしか許されない

TLE をどう回避するか？
 $N = 10^5$ で計算量 $O(N^2)$ のプログラムを書くと、計算回数が 10^{10} 程度となり、数秒以内に実行が終わらず TLE となってしまう

2 小課題 2 | $N = M$ (高速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- そこで、あらかじめ、絵を座標の小さい順に並び替えておくことを考える

2 小課題 2 | $N = M$ (高速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● そこで、あらかじめ、絵を座標の小さい順に並び替えておくことを考える

C++ では `std::sort` 関数を使うと実装できる

計算量は $O(N \log N)$ です、知らない人は是非調べてみましょう

2 小課題 2 | $N = M$ (高速な解法)

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 最も距離が短いのは隣り合う 2 つの絵であるはずなので、
 $i = 1, 2, \dots, N - 1$ について「左から i 番目と $i + 1$ 番目の距離が K 以上かどうか」をチェックすれば良い

2 小課題 2 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ



このような場合を考える
絵間距離の最小値はいくつか？

2 小課題 2 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ



$i = 1$ の場合: 絵 1 と絵 2 の距離は $|4 - 1| = 3$

現在の距離の最小値 = 3

2 小課題 2 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ



$i = 2$ の場合: 絵 2 と絵 3 の距離は $|6 - 4| = 2$

現在の距離の最小値 = 2

2 小課題 2 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ



$i = 3$ の場合: 絵 3 と絵 4 の距離は $|8 - 6| = 2$

現在の距離の最小値 = 2

2 小課題 2 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

こんな感じで距離の最小が 2 と分かる

これが K 未満であれば答えは「-1」

現在の距離の最小値 = 2

2 小課題 2 | 実装例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

たとえば次のような実装が考えられる

実装例 (C++)

```
#include <bits/stdc++.h>
using namespace std;

int N, M, D, Answer = (1 << 30);
int X[200009], V[200009];

bool check() {
    for (int i = 1; i <= N - 1; i++) {
        if (X[i + 1] - X[i] < D) return false;
    }
    return true;
}
```

```
int main() {
    cin >> N >> M >> D;
    for (int i = 1; i <= N; i++) {
        cin >> X[i] >> V[i];
        Answer = max(Answer, V[i]);
    }
    sort(X + 1, X + N + 1);

    if (check() == true) cout << Answer << endl;
    else cout << "-1" << endl;
    return 0;
}
```

15点

35

目次

はじめに

1 ページ

小課題 1 ($M = 1$)、小課題 2 ($N = M$)

15 ページ

小課題 3 ($N \leq 15$)

36 ページ

小課題 4・5／満点解法

50 ページ

まとめ

80 ページ

3

小課題 3

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 制約は $N \leq 15$ とかなり小さい

3

小課題 3

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 制約は $N \leq 15$ とかなり小さい

絵の残し方を全通り調べ上げる「全探索アルゴリズム」が通用するのではないか？

● 制約は $N \leq 15$ とかなり小さい

絵の残し方を全通り調べ上げる「全探索アルゴリズム」が通用するのではないか？

絵の残し方は 2^N 通り存在するため、 $N = 15$ では高々 32,768 通り。判定パートの計算量は小課題 2 で説明した通り $O(N \log N)$ なので、余裕を持って実行時間に間に合う。

3

小課題 3 | 全探索の実装方法

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 全探索の実装方法として、例えば次のようなものが考えられる

実装例 (C++)

```
bool check() {  
    if (N == 15) {  
        for (int i1 = 0; i1 <= 1; i1++) {  
            for (int i2 = 0; i2 <= 1; i2++) {  
                for (int i3 = 0; i3 <= 1; i3++) {  
                    for (int i4 = 0; i4 <= 1; i4++) {  
                        :  
                    }  
                }  
            }  
        }  
    }  
}
```

t 個目の絵を選ぶ場合は $i_t = 1$

選ばない場合は $i_t = 0$

N の値で場合分けして N 重ループを書く

3

小課題 3 | 全探索の実装方法

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

全探索の実装方法として、例えば次のようなものが考えられる

実装例 (C++)

```
bool check(  
    if (N == 0) {  
        for (int i1 = 0; i1 <= 1; i1++) {  
            for (int i2 = 0; i2 <= 1; i2++) {  
                for (int i3 = 0; i3 <= 1; i3++) {  
                    for (int i4 = 0; i4 <= 1; i4++) {  
                        :  
                    }  
                }  
            }  
        }  
    }  
}
```

実装が大変になってしまおう！

どうしよう？？？

i_t 個目の箱を選ぶ場合は $i_t = 1$

選ばない場合は $i_t = 0$

N の値で場合分けして N 重ループを書く

3

小課題 3 | より良い実装

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 「bit 全探索」という手法を使う

3 小課題 3 | より良い実装

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 「bit 全探索」という手法を使う

$i = 0, 1, 2, 3, \dots, 2^N - 1$ とループを回して i を 2 進数で表したときに

下から j 桁目が 1 である場合 j 個目の絵を選び、そうでなければ選ばない

3 小課題 3 | より良い実装

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

「bit 全探索」という手法を使う

$i = 0, 1, 2, 3, \dots, 2^N - 1$ とループを回して i を 2 進数で表したときに

下から j 桁目が 1 である場合 j 個目の絵を選び、そうでなければ選ばない

i の値	0	1	2	3	4	5	6	7
2 進数の値	000	001	010	011	100	101	110	111
選ぶ絵	{}	{1}	{2}	{1,2}	{3}	{1,3}	{2,3}	{1,2,3}

3 小課題 3 | より良い実装

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 整数 i を 2 進数で表したときの下から $j + 1$ 桁目の値は、
ビット演算を用いて次のように計算できる

3

小課題 3 | より良い実装

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 整数 i を 2 進数で表したときの下から $j + 1$ 桁目の値は、
ビット演算を用いて次のように計算できる

└─ $(i \text{ and } 2^j) = 0$ 、つまり「 $(i \ \& \ (1 \ll j)) == 0$ 」のとき、 $j + 1$ 桁目は 0

└─ $(i \text{ and } 2^j) \neq 0$ 、つまり「 $(i \ \& \ (1 \ll j)) \neq 0$ 」のとき、 $j + 1$ 桁目は 1

3

小課題 3 | より良い実装

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 例えば以下のようなコードで 2 進数表記の j 桁目の値がわかる

実装例 (C++)

```
void binary_henkan(int x) {  
    int bit[15];  
    for (int i = 0; i < 15; i++) {  
        if ((x & (1 << i)) == 0) bit[i] = 0;  
        if ((x & (1 << i)) != 0) bit[i] = 1;  
    }  
}
```

$\text{bit}[i]$ は 2 進数での $i + 1$ 桁目

例えば $x = 9$ の場合

$\text{bit} = [1, 0, 0, 1, 0, 0, 0, \dots]$

3

小課題 3 | 解法まとめ

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● まとめると、 2^N 通りの方法を全探索することで、計算量 $O(2^N \times N^2)$ または $O(2^N \times N \log N)$ で解ける

└ 判定パートは小課題 2 の通り、 $O(N^2)$ または $O(N \log N)$ かかる

└ 実装のひとつの方法として「bit 全探索」というものが挙げられる

3

小課題 3 | 実装例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

たとえば次のような実装が考えられる

実装例 (C++)

```
#include <bits/stdc++.h>
using namespace std;

int N, M, D, Answer = -1;
int X[200009], V[200009];

int check(vector<int> x) {
    // {x[0], x[1], ...} の選び方が OK かの判定
    // OK の場合価値の最小を返す (詳細の実装は略)
}

int main() {
    cin >> N >> M >> D;
```

```
    for (int i = 0; i < N; i++) {
        cin >> X[i] >> V[i];
    }

    for (int i = 0; i < (1 << N); i++) {
        vector<int> vec;
        for (int j = 0; j < N; j++) {
            if ((i & (1 << j)) != 0) {
                vec.push_back(j);
            }
        }
        Answer = max(Answer, check(vec));
    }
    cout << Answer << endl;
    return 0;
}
```

34点

目次

はじめに

1 ページ

小課題 1 ($M = 1$)、小課題 2 ($N = M$)

15 ページ

小課題 3 ($N \leq 15$)

36 ページ

小課題 4・5／満点解法

50 ページ

まとめ

80 ページ

4 小課題 4 | $V_i \leq 2$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- $V_i \leq 2$ の場合、答えは「2」「1」「存在しない」のいずれか
したがって、次のような問題が解ければよい

4 小課題 4 | $V_i \leq 2$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4~6

5.まとめ

● $V_i \leq 2$ の場合、答えは「2」「1」「存在しない」のいずれか
したがって、次のような問題が解ければよい

価値 2 以上の絵のみを使って M 個以上の絵を残せるか？

価値 1 以上の絵のみを使って M 個以上の絵を残せるか？

どれにも当てはまらなければ、答えは「存在しない(-1)」

4 小課題 4 | $V_i \leq 2$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4~6

5.まとめ

● $V_i \leq 2$ の場合、答えは「2」「1」「存在しない」のいずれか

したがって、次のような問題が解ければよい

ではどうやって M 個以上残せるか

価値 2 以上の絵のみを使って M 個以上の絵を残せるか？

判定すればよいのか？

価値 2 以上の絵のみを使って M 個以上の絵を残せるか？

どれにも当てはまらなければ、答えは「存在しない(-1)」

4 小課題 4 | $V_i \leq 2$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4~6

5.まとめ

- 左にある「価値が B 以上のもの」から順に、貪欲に取っていけば良い

4 小課題 4 | $V_i \leq 2$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4~6

5.まとめ

● 左にある「価値が B (1 または 2) 以上のもの」から順に、貪欲に取っていけば良い

└─ 直前に取ったものから距離が D 未満である場合、この絵を取らない

└─ 直前に取ったものから距離が D 以上である場合、この絵を取る

4 小課題 4 | 具体例

1.はじめに

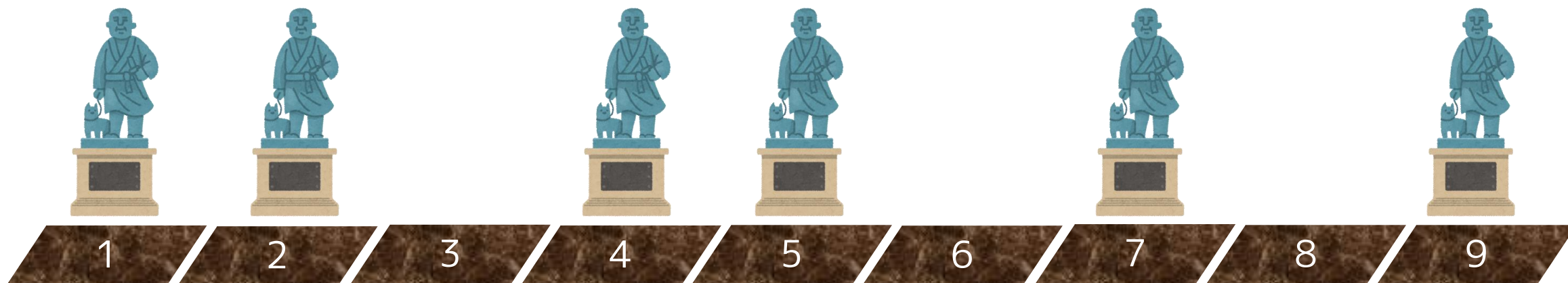
2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



直前に残した絵は存在しない

→ この絵は残せる

4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



直前に残した絵と距離 1 しか離れていない

→ この絵は残せない

4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



直前に残した絵と距離 3 しか離れていない

→ この絵は残せない

4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



直前に残した絵と距離 4 離れている

→ この絵は残せる

4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



直前に残した絵と距離 2 しか離れていない

→ この絵は残せない

4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)



直前に残した絵と距離 4 離れている

→ この絵は残せる

4 小課題 4 | 具体例

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● $M = 3$ 個以上の「価値 1 以上の絵」を取れるか？ ($D = 4$ とする)

合計 3 つの絵を残せる

こんな感じで貪欲に選べばよい！

直前に残した絵と距離 4 離れている

→ この絵は残せる

4 小課題 4 | $V_i \leq 2$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- これを価値の下限 $B = 2, B = 1$ について調べればよい
実装例は次の通り ($X_1 < X_2 < \dots < X_N$ の場合)

実装例 (C++)

```
#include <bits/stdc++.h>
using namespace std;
int N, M, D, X[200009], V[200009];

bool check(int B) {
    int pre = -1000000000, cnt = 0;
    for (int i = 1; i <= N; i++) {
        if (V[i] < B || X[i] - pre < D) continue;
```

```
        cnt += 1; pre = X[i];
    }
    if (cnt < M) return false;
    return true;
}

int main() {
    // 入力省略
    if (check(2) == true) printf("2¥n");
    else if (check(1) == true) printf("1¥n");
    else printf("-1¥n");
    return 0;
}
```

51点

64

4 小課題 5 | $N \leq 1000$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4~6

5.まとめ

● 実は、答え（価値の最小）は $V_1, V_2, \dots, V_N$ のうちいずれかである

4 小課題 5 | $N \leq 1000$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4~6

5.まとめ

● 実は、答え（価値の最小）は $V_1, V_2, \dots, V_N$ のうちいずれかである

価値が V_1 以上のものだけで M 個以上選べないか？

価値が V_2 以上のものだけで M 個以上選べないか？

価値が V_N 以上のものだけで M 個以上選べないか？

選べたものの中で価値の下限が最も大きかったものが答え

4 小課題 5 | $N \leq 1000$ の場合

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● N 回 check 関数を呼び出すので、全体計算量は $O(N^2)$

実装例 (C++)

```
#include <bits/stdc++.h>
using namespace std;
int N, M, D, X[200009], V[200009], Answer = -1;

// check 関数は省略
int main() {
    // 入力は省略
    for (int i = 1; i <= N; i++) {
        if (check(V[i]) == true) Answer = max(Answer, V[i]);
    }
    cout << Answer << endl;
    return 0;
}
```

73点

67

4

満点解法

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 小課題 5 と同様に N 回 check 関数を呼び出すと、計算量が $O(N^2)$ となり TLE してしまう

どうしよう ???

4

満点解法

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 小課題 5 と同様に N 回 check 関数を呼び出すと、計算量が $O(N^2)$ となり TLE してしまう

最小値の最大化は

答えで二分探索

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 最初、答えが 1 以上 16 以下であることが分かっていたとする

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 最初、答えが 1 以上 16 以下であることが分かっていたとする

現在の答えの範囲	質問	関数呼び出し	結果例
1～16	答えは 9 以上か？	check(9)	Yes

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 最初、答えが 1 以上 16 以下であることが分かっていたとする

現在の答えの範囲	質問	関数呼び出し	結果例
1～16	答えは 9 以上か？	check(9)	Yes
9～16	答えは 13 以上か？	check(13)	No

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 最初、答えが 1 以上 16 以下であることが分かっていたとする

現在の答えの範囲	質問	関数呼び出し	結果例
1～16	答えは 9 以上か？	check(9)	Yes
9～16	答えは 13 以上か？	check(13)	No
9～12	答えは 11 以上か？	check(11)	No

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 最初、答えが 1 以上 16 以下であることが分かっていたとする

現在の答えの範囲	質問	関数呼び出し	結果例
1～16	答えは 9 以上か？	check(9)	Yes
9～16	答えは 13 以上か？	check(13)	No
9～12	答えは 11 以上か？	check(11)	No
9～10	答えは 10 以上か？	check(10)	Yes

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 最初、答えが 1 以上 16 以下であることが分かっていたとする

現在の答えの範囲	質問	関数呼び出し	結果例
1～16	答えは 9 以上か？	check(9)	Yes
9～16	答えは 13 以上か？	check(13)	No
9～12	答えは 11 以上か？	check(11)	No
9～10	答えは 10 以上か？	check(10)	Yes
10	-	-	-

4 満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 最初、答えが 1 以上 16 以下であることが分かっていたとする

現在の答えの範囲	質問	関数呼び出し	結果例
1~16	答えは 9 以上か？	check(9)	Yes
9~16	答えは 13 以上か？	check(13)	No
9~12	答えは 11 以上か？	check(11)	No
9~10	答えは 10 以上か？	check(10)	Yes
10	-	-	-

答えの範囲を半分に絞ることで

4 回の関数呼出で答えが分かる！

4 満点解法 | 答えで二分探索

1.はじめに

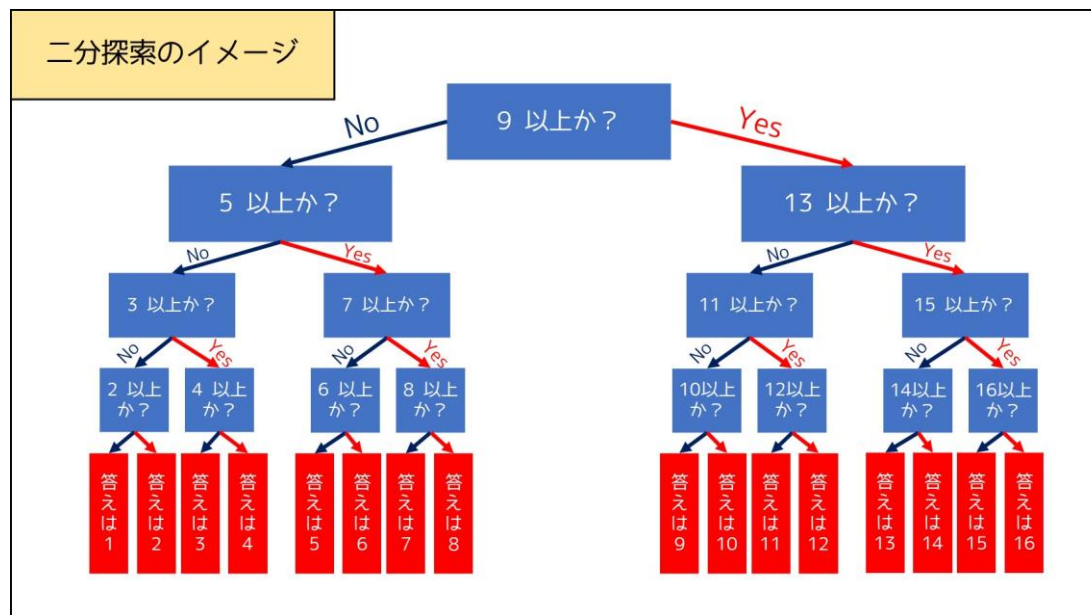
2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

● 全部まとめて図で表すこんな感じ



<https://qiita.com/e869120/items/b4a0493aac567c6a7240> より引用

4 満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- a 回 check 関数を呼び出すと、探索範囲が 2^a 分の 1 に絞られる
答えの最大値を L として、探索範囲が L 分の 1 以下になれば答えが一通りに定まるので、

$$2^a \geq L (\Leftrightarrow) a \geq \log_2 L$$

つまり、 $O(\log_2 L)$ 回 check 関数を呼び出せばよい

4

満点解法 | 答えで二分探索

1.はじめに

2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

- 全体としては、答えで二分探索をすることで現実的な時間で実行が終わる。計算量は $O(N (\log N + \log L))$ となる。

100
点

目次

はじめに

1 ページ

小課題 1 ($M = 1$)、小課題 2 ($N = M$)

15 ページ

小課題 3 ($N \leq 15$)

36 ページ

小課題 4・5／満点解法

50 ページ

まとめ

80 ページ

● 各小課題において、次のような解法で解くことができる

小課題	制約	キーワード
1	$M = 1$	$\max(V_1, V_2, \dots, V_N)$ が答え
2	$N = M$	絵を座標の小さい順にソートしてチェックする
3	$N \leq 15$	bit 全探索をしよう
4	$N \leq 1000, V_i = 2$	「答えは2か？」 → 「答えは1か？」
5	$N \leq 1000$	答えは $V_1, V_2, \dots, V_N$ のいずれかなので全探索する
6	$N \leq 10^5$	答えで二分探索

5

得点分布

1.はじめに

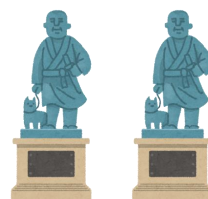
2.小課題 1・2

3.小課題 3

4.小課題 4～6

5.まとめ

100点



50～99点



20～49点



1～19点





ご清聴ありがとうございました

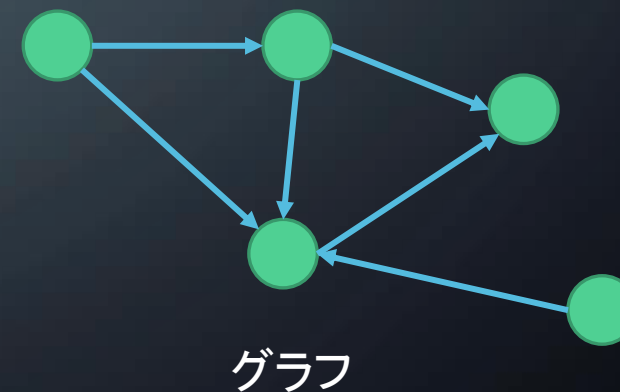
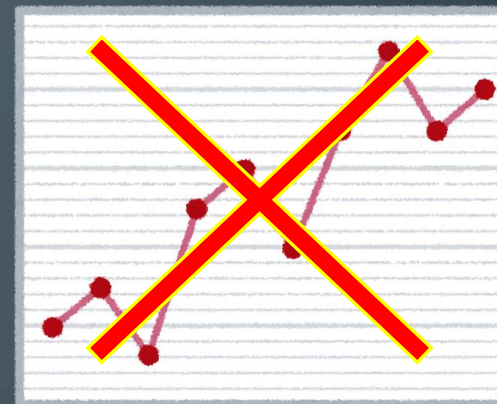
パレード 解説

JOIG2021 5問目



問題概要

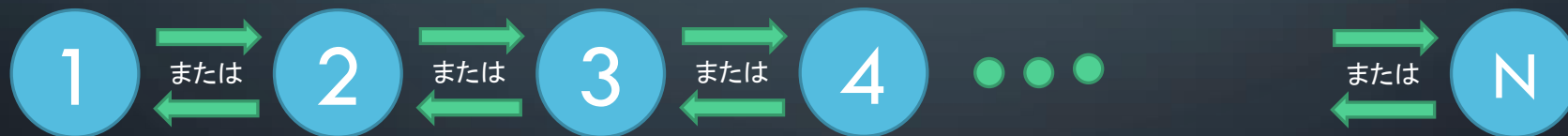
- N 頂点 M 辺の重み付き有向グラフが与えられる
- 頂点 1 から頂点 N まで距離 L 以下で移動する
- 進行方向を反転しなければならない辺の本数の最小値は？
- 到達不可能な場合はそれを判定



小課題 1 (7点)

- $M = N-1$
- $(A_i, B_i) = (i, i+1)$ または $(A_i, B_i) = (i+1, i)$

この制約が意味するものは？



小課題 1 (7点)



- 頂点 1 から頂点 N までの距離は 全ての辺の長さの合計
- 反転する辺の数は $(A_i, B_i) = (i+1, i)$ となっている辺の数

これらをfor文など、適切な繰り返し処理を用いて計算しましょう



小課題 2 (14点)

- M は偶数
- $A_{2i-1} = B_{2i}$
- $A_{2i} = B_{2i-1}$
- $C_{2i-1} = C_{2i}$

この制約が意味するものとは？

→ 辺が (実質) 双方向

小課題 2 (14点)

辺が双方向だと何が嬉しいか？

→ 反転を考えなくてよい, 到達可能か不可能かを判定すればよい

→ 頂点 1 から頂点 N までの最短路が L 以下か？



小課題 2 (14点)

グラフ上の最短路を求める方法の例

- Bellman–Ford algorithm (ベルマンフォード法)
- Dijkstra's algorithm (ダイクストラ法)
- Warshall–Floyd Algorithm (ワーシャルフロイド法)

問題設定に合った適切なアルゴリズムを使おう

この問題の場合, ワーシャルフロイド法だと実行時間制限超過(TLE)するかも

小課題 3 (18点)

- $N \leq 15$
- $M \leq 15$

M が非常に小さい

→各辺について, 反転させるかさせないか試す(bit全探索)

小課題 3 (18点) (別解)

- $N \leq 15$
- $M \leq 15$

M が非常に小さい

→ 深さ優先探索 (DFS) など で 全経路 を 調べる



小課題 4 (20点)

- $C_i = 1$
- $L = 1000000000$

この制約が意味するものは？

→辺の長さについて考えなくてよい



小課題 4 (20点)

辺の長さについて考えなくてよい

→反転の回数だけ考えればよい

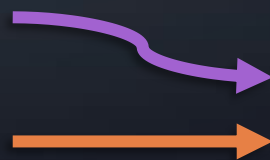
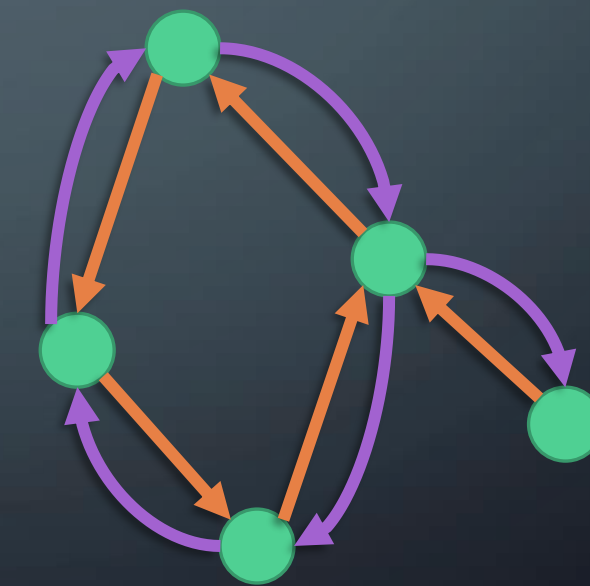
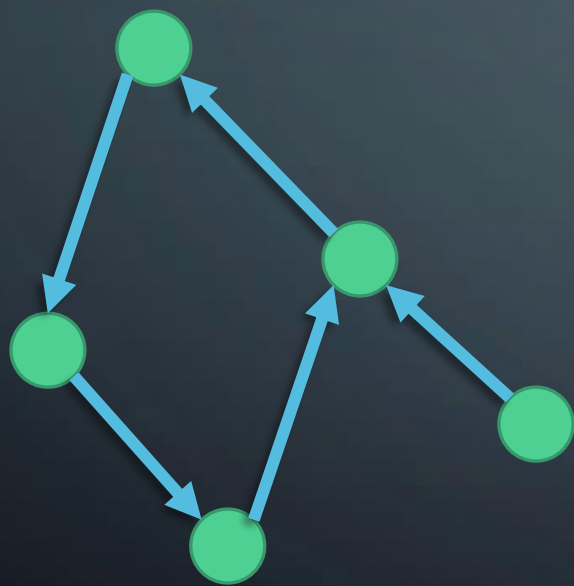
→辺の進行方向にコスト0, 反転方向にコスト1の辺を貼って最短路を求める

ダイクストラ法やベルマンフォード法でも正解できるが, 01-BFSという手法も使える



小課題 4 (20点)

辺の進行方向にコスト0, 反転方向にコスト1の辺を貼って最短路を求める



コスト1
コスト0



小課題 5 (20点)

- $C_i = 1$

小課題 4 と異なり, L の制約がない

→ちゃんと辺の長さ, 各頂点までの距離について考えないといけない

→距離の最大値は $N-1$ (各頂点に2回以上訪れる経路は最善ではないため)

小課題 5 (20点)

距離の最大値は $N-1$ (各頂点に2回以上訪れる経路は最善ではないため)

→ (頂点, この頂点に訪れるまでの距離) の組み合わせは N^2 通り

→ この N^2 頂点に適切に長さ 0 か 1 の辺を貼り, 最短路問題を解く

距離の小さい順にみてやることで動的計画法 (DP) みたいにも解ける

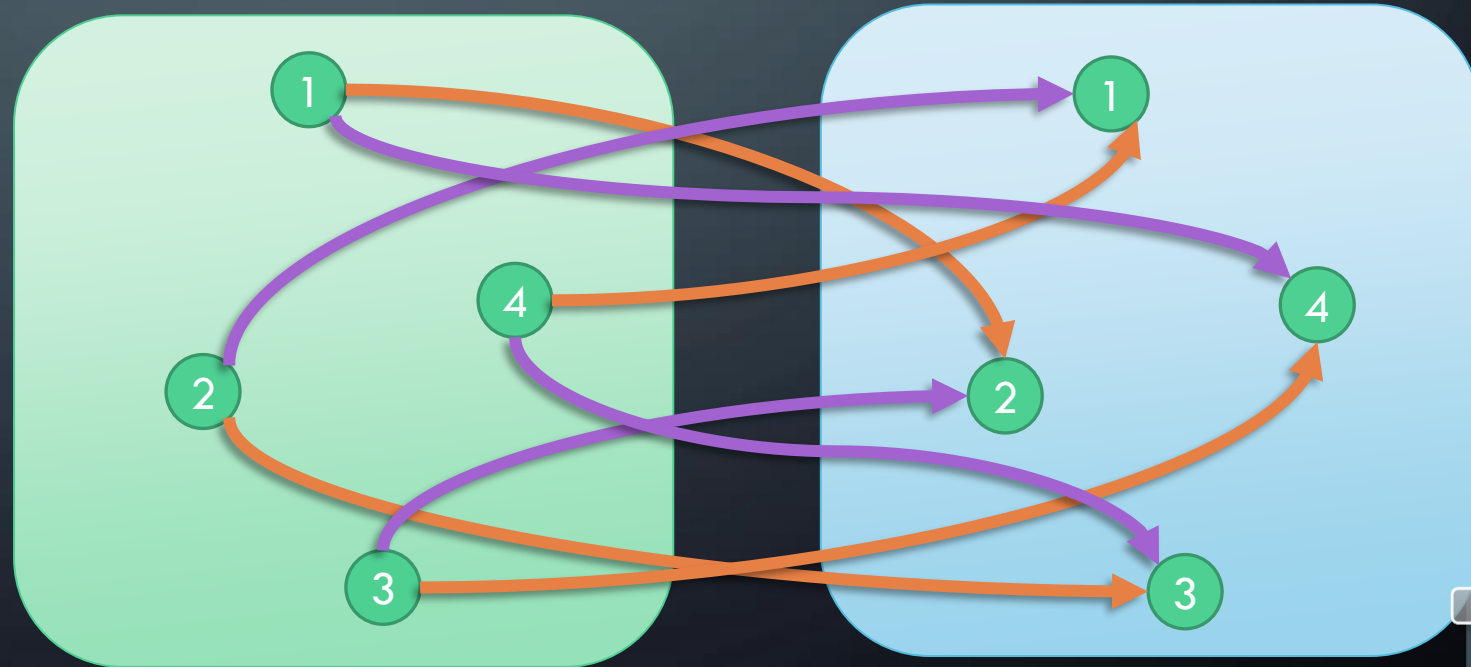
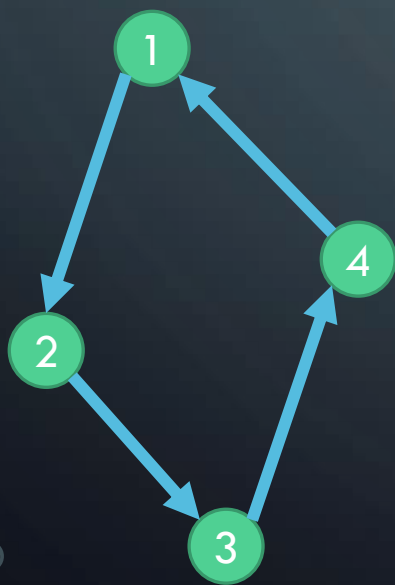
小課題 5 (20点)

(頂点, この頂点に訪れるまでの距離) の組み合わせは N^2 通り

→ この N^2 頂点に適切に長さ 0 か 1 の辺を貼り, 最短路問題を解く

コスト1

コスト0



満点解法

小課題 5 では(頂点, 距離)をindexとして扱った

ここで, 反転させる道の個数を考えると最大で $N-1$ 本

→ (頂点, この頂点にたどり着くために反転させた本数)をindexにする

→ 小課題 5 と同様にこの N^2 頂点に適切に辺を貼り, 最短路問題を解く

遅い言語だと $O(NM)$ 本辺を貼ると実行時間制限超過するかもしれない

反転させた本数の小さい順に見てやると多少早くなる



得点分布

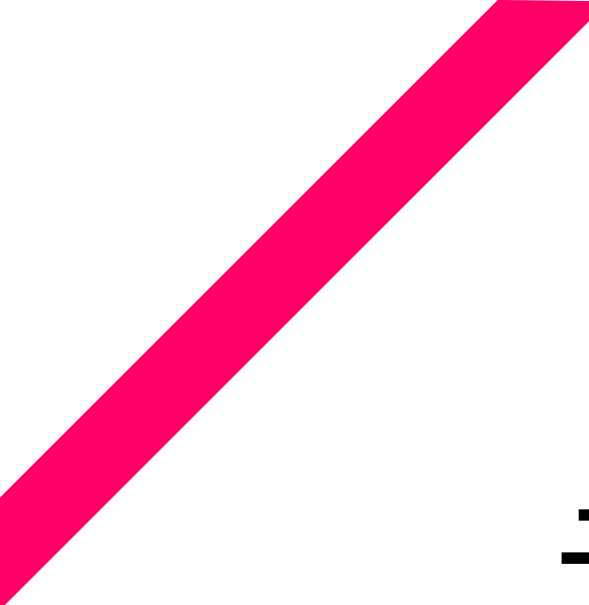
0点 117人

7点 4人

20点 1人

21点 2人

25点 1人



デジタルアート 解説

日本情報オリンピック女性部門 (JOIG 2021) 問題 6
2021. 4. 25

解説：米田 寛峻

はじめに | 問題概要

はじめに

1

2

3

4

5

6

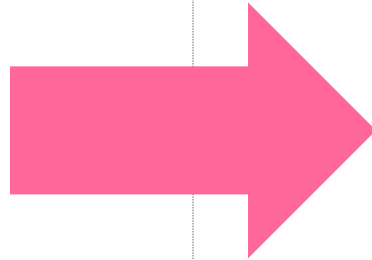
7

8

満点

Page 2 of 54

1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1



1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3					3
1	2	2	2					1
1	2	2	2					1
1	2	2	2					1
1	2	2	2					1

$H \times W$ ピクセルの 画像 がある

s ピクセル以下の 長方形領域
を選んで隠す

はじめに | 問題概要

はじめに

1

2

3

4

5

6

7

8

満点

Page 3 of 54

隠されず見える色の種類を小さくしたい
「最小で何種類にできるか？」

$H \times W$ ピクセルの 画像 がある

S ピクセル以下の 長方形領域
を選んで隠す

はじめに | 制約

はじめに

1

2

3

4

5

6

7

8

満点

Page 4 of 54

小課題ごとに「入力データの制約」が変わってくる

→ 解法の性能に応じて得点が決められる！

小課題	配点	入力データの制約
#1	8 点	$H = 1, W \leq 10$
#2	10 点	$H \leq 10, W \leq 10$
#3	5 点	$S = 1$
#4	6 点	$1 \leq A_{i,j} \leq 2$
#5	5 点	$1 \leq A_{i,j} \leq 4$
#6	13 点	$1 \leq A_{i,j} \leq 15$
#7	13 点	$1 \leq A_{i,j} \leq 30$
#8	15 点	$1 \leq A_{i,j} \leq 70$
#9	25 点	追加の制約はない.

全体の制約

✂ $1 \leq H \leq 1000$

✂ $1 \leq W \leq 1000$

✂ $1 \leq S \leq HW$

✂ $1 \leq A_{i,j} \leq 256$

※ $A_{i,j}$ はピクセル (i,j) の色の番号を表す

はじめに | 制約

はじめに

1

2

3

4

5

6

7

8

満点

Page 5 of 54

小課題ごとに「入力データの制約」が変わってくる

→ 解法の性能に応じて得点が決められる！

小課題	配点	入力データの制約
#1	8 点	$H = 1, W \leq 10$
#2	10 点	$H \leq 10, W \leq 10$
#3	5 点	$S = 1$
#4	6 点	$1 \leq A_{i,j} \leq 2$
#5	5 点	$1 \leq A_{i,j} \leq 4$
#6	13 点	$1 \leq A_{i,j} \leq 15$
#7	13 点	$1 \leq A_{i,j} \leq 30$
#8	15 点	$1 \leq A_{i,j} \leq 70$
#9	25 点	追加の制約はない.

1. H, W に関する制約

✂ 小課題 1, 2 が関係する

→ ここまで 8 点

→ ここまで 18 点

はじめに | 制約

はじめに

1

2

3

4

5

6

7

8

満点

Page 6 of 54

小課題ごとに「入力データの制約」が変わってくる

→ 解法の性能に応じて得点が決められる！

小課題	配点	入力データの制約
#1	8 点	$H = 1, W \leq 10$
#2	10 点	$H \leq 10, W \leq 10$
#3	5 点	$S = 1$
#4	6 点	$1 \leq A_{i,j} \leq 2$
#5	5 点	$1 \leq A_{i,j} \leq 4$
#6	13 点	$1 \leq A_{i,j} \leq 15$
#7	13 点	$1 \leq A_{i,j} \leq 30$
#8	15 点	$1 \leq A_{i,j} \leq 70$
#9	25 点	追加の制約はない.

2. S に関する制約

✖ 小課題 3 が関係する

✖ 小課題 2 と合わせて取ると
23 点 が取れる

→ ここまで 5 点

はじめに | 制約

はじめに

1

2

3

4

5

6

7

8

満点

Page 7 of 54

小課題ごとに「入力データの制約」が変わってくる

→ 解法の性能に応じて得点が決める！

小課題	配点	入力データの制約
#1	8 点	$H = 1, W \leq 10$
#2	10 点	$H \leq 10, W \leq 10$
#3	5 点	$S = 1$
#4	6 点	$1 \leq A_{i,j} \leq 2$
#5	5 点	$1 \leq A_{i,j} \leq 4$
#6	13 点	$1 \leq A_{i,j} \leq 15$
#7	13 点	$1 \leq A_{i,j} \leq 30$
#8	15 点	$1 \leq A_{i,j} \leq 70$
#9	25 点	追加の制約はない.

3. $A_{i,j}$ に関する制約

- ✖ 小課題 4～8 が関係する
- ✖ 全体の制約も「 $1 \leq A_{i,j} \leq 256$ 」
小課題 8 の延長線になっている

→ ここまで 6 点

→ ここまで 11 点

→ ここまで 24 点

→ ここまで 37 点

→ ここまで 52 点

→ これが解ければ 100 点満点

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 8 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 1 | $H = 1, W \leq 10$

はじめに

1

2

3

4

5

6

7

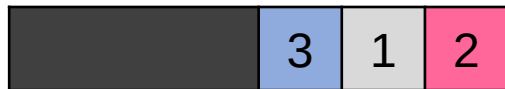
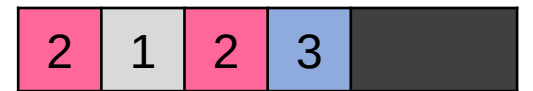
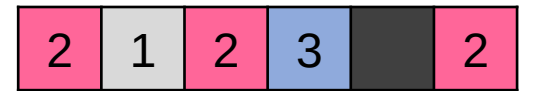
8

満点

Page 9 of 54

小課題 1 では、画像は $1 \times W$ の形になっている

画像の隠し方を全部試してみよう（例： $H = 1, W = 6, S = 4$ の場合）



最小で 1 種類

小課題 1 | $H = 1, W \leq 10$

はじめに

1

2

3

4

5

6

7

8

満点

Page 10 of 54

このように、全ての隠し方について試す「全探索」で解ける

色の種類数は、「出現が 1 回以上」である色の個数として求められる

- 番号 $1, 2, \dots, 256$ の色の出現回数 $c_1, c_2, \dots, c_{256}$ をカウントすると実装しやすい

3 重ループを実装することで、答えを求めるプログラムができる

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 11 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 2 | $H \leq 10, W \leq 10$

はじめに

1

2

3

4

5

6

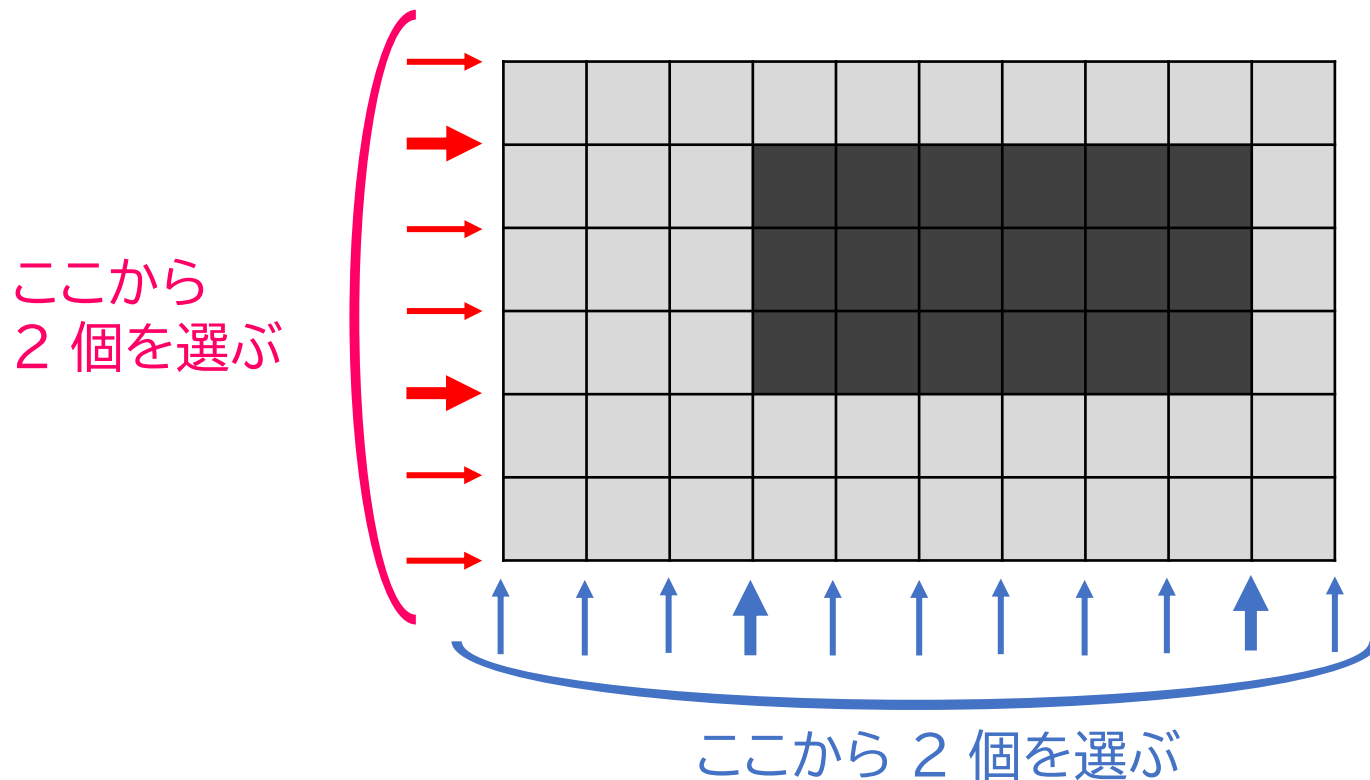
7

8

満点

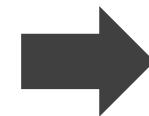
Page 12 of 54

小課題 2 は、画像のサイズが小さいため、**全探索** が使えそう
実際に、隠す長方形領域は何通りあるのか？



縦の選び方 $\cdots \frac{1}{2}H(H+1)$ 通り

横の選び方 $\cdots \frac{1}{2}W(W+1)$ 通り



全体の選び方は

$\frac{1}{4} \times H(H+1) \times W(W+1)$ 通り

小課題 2 | $H \leq 10, W \leq 10$

はじめに

1

2

3

4

5

6

7

8

満点

Page 13 of 54

小課題 1 と同様に、全ての隠し方について試す「全探索」で解ける

- 1 つのの長方形領域に対して種類数を $HW + A$ 回程度の計算の手間をかけると…
- 計算時間 $O(H^2W^2 \times (HW + A))$ で解くことができる

※ A を色の番号の最大値、すなわち $A = 256$ とする

6 重ループ を実装する必要があり、プログラム作成の難易度が比較的高い

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 14 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 3 | $s = 1$

はじめに

1

2

3

4

5

6

7

8

満点

Page 15 of 54

小課題 3 では「1 ピクセルだけ隠すことになる」

	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

1	1		3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

1	1	3	3		3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4		1



1		3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

1	1	3		3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

1	1	3	3	3		3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

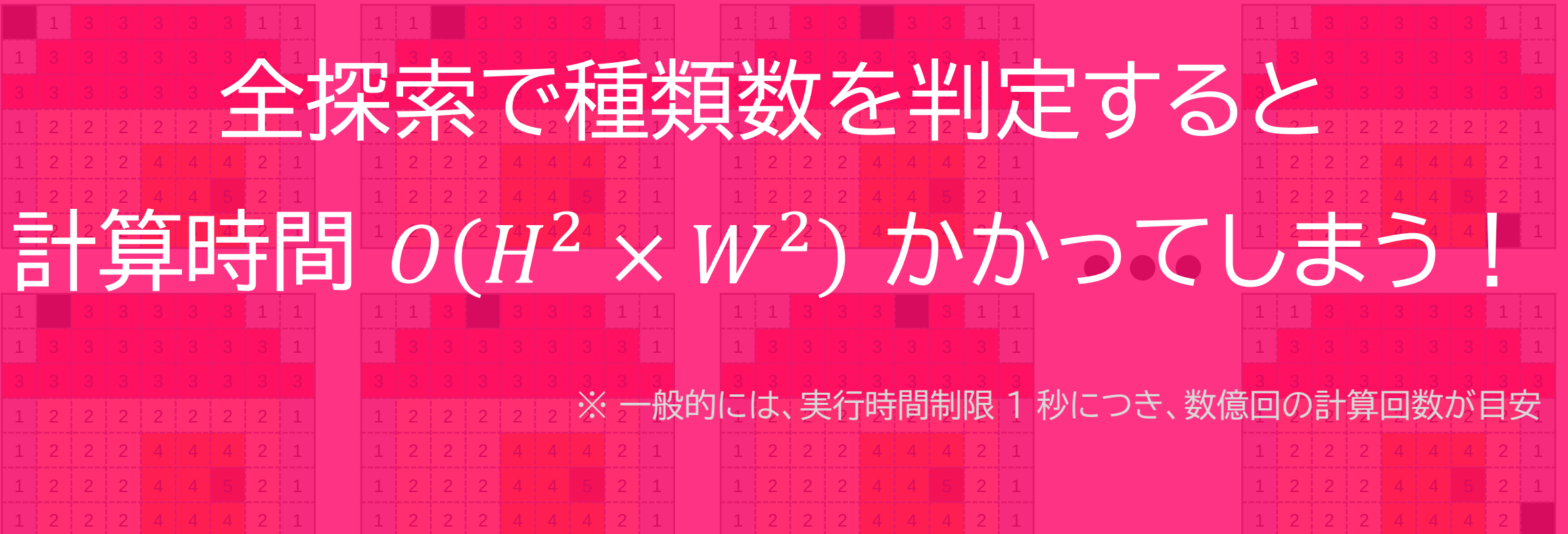
1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	

小課題 3 | $s = 1$

小課題 3 では「1 ピクセルだけ隠すことになる」

全探索で種類数を判定すると
計算時間 $O(H^2 \times W^2)$ がかかってしまう！

※ 一般的には、実行時間制限 1 秒につき、数億回の計算回数が目安



小課題 3 | $s = 1$

はじめに

1

2

3

4

5

6

7

8

満点

Page 17 of 54

1 ピクセル隠すことで 種類数を減らせるか 考える

1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1		2	2	4	4	4	2	1
1	2	2	2	4	4	5	2	1
1	2	2	2	4	4	4	2	1

種類数が 5 のままの例

1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4		2	1
1	2	2	2	4	4	4	2	1

種類数が 5 → 4 になる例

小課題 3 | $s = 1$

はじめに

1

2

3

4

5

6

7

8

満点

Page 18 of 54

1 ピクセル隠すことで種類数を減らせるか考える

右図の例で、種類数を減らせるのは

- 1 回しか登場しない色を消したから

種類数が 1 減らせる条件は…

条件 「出現が 1 回」の色があること

1	1	3	3	3	3	3	1	1
1	3	3	3	3	3	3	3	1
3	3	3	3	3	3	3	3	3
1	2	2	2	2	2	2	2	1
1	2	2	2	4	4	4	2	1
1	2	2	2	4	4		2	1
1	2	2	2	4	4	4	2	1

種類数が 5 → 4 になる例

小課題 3 | $S = 1$

はじめに

1

2

3

4

5

6

7

8

満点

Page 19 of 54

アルゴリズム

- 1 各色の出現回数 $c_1, c_2, \dots, c_{256}$ を求める
- 2 隠さない場合の種類数 T は「 $c_1, c_2, \dots, c_{256}$ の中で 0 以外の数の個数」で求まる
- 3 $c_1, c_2, \dots, c_{256}$ の中に 1 が **あれば** 答えは $T - 1$ となる
 $c_1, c_2, \dots, c_{256}$ の中に 1 が **なければ** 答えは T となる

計算時間 $O(HW + A)$ で高速に動作する

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 20 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 4 | 2 色だけの場合

はじめに

1

2

3

4

5

6

7

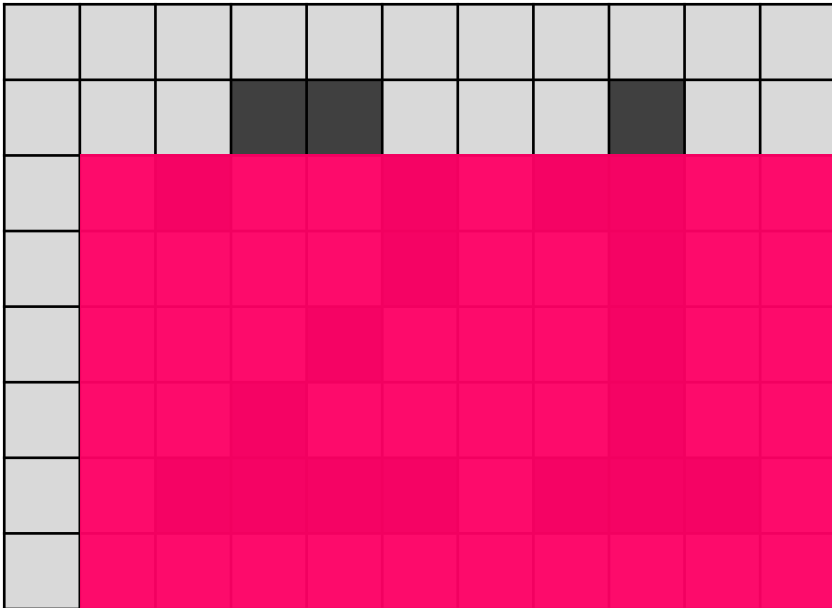
8

満点

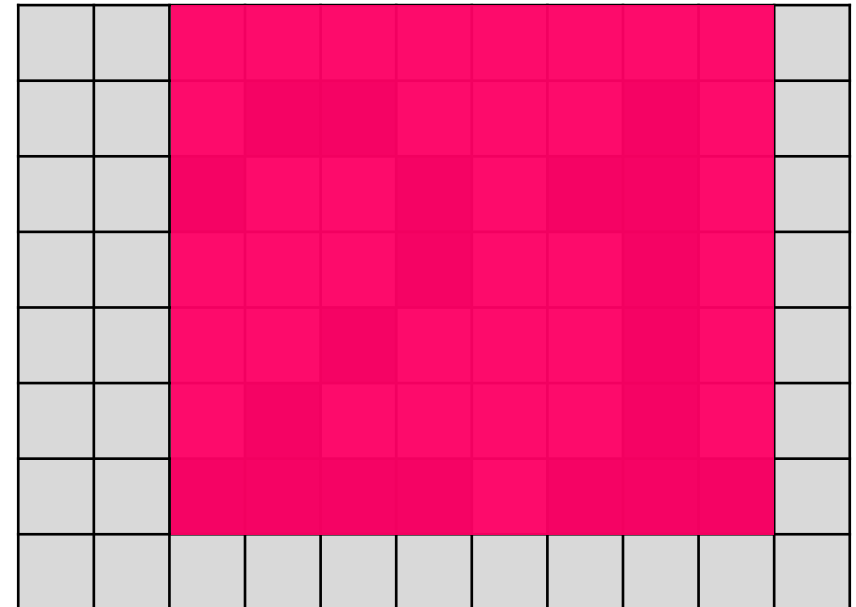
Page 22 of 54

Q. 黒のエリアを隠して見えなくすることはできるか？ ($s = 56$ の場合)

× 隠せていない場合



○ 隠せている場合



小課題 4 | 2 色だけの場合

はじめに

1

2

3

4

5

6

7

8

満点

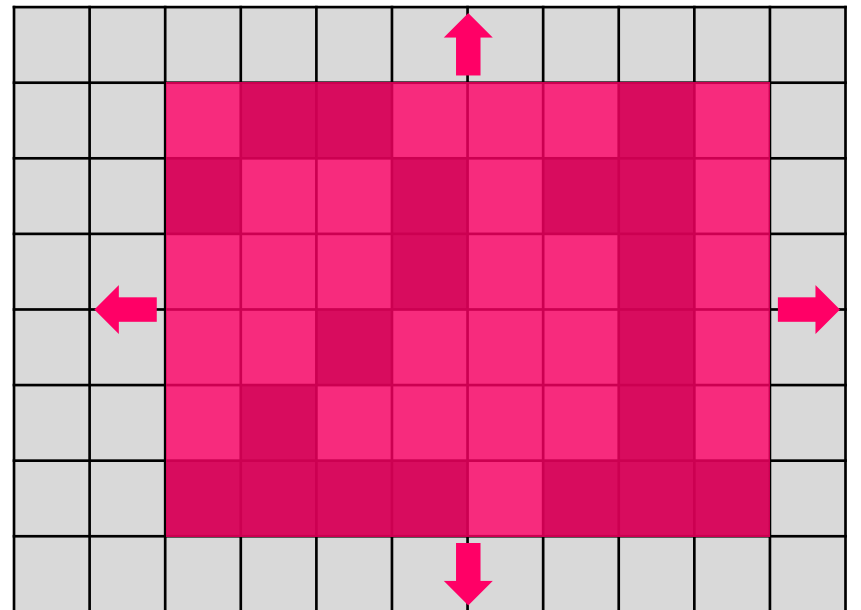
Page 23 of 54

すべて包み込む「ギリギリの長方形」が、隠せる限界となる

ギリギリの長方形 の計算

- 上端：黒マスの中で一番上の位置 ra
- 下端：黒マスの中で一番下の位置 rb
- 左端：黒マスの中で一番左の位置 ca
- 右端：黒マスの中で一番右の位置 cb

必要最小限の面積は $(rb - ra + 1) \times (cb - ca + 1)$



小課題 4 | 2 色だけの場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 24 of 54

アルゴリズム：黒のエリアを全部隠せるか判定

- 1 黒マスの最も上の位置 ra 、最も下の位置 rb を求める
- 2 黒マスの最も左の位置 ca 、最も下の位置 cb を求める
- 3 $(rb - ra + 1) \times (cb - ca + 1) \leq S$ ならば「すべて隠せる」
そうでないなら「すべて隠すことはできない」

※ 黒マスが 1 個もない場合は、例外的に「全部隠せる」とする

計算時間 $O(HW)$ で高速に動作する

小課題 4 | 2 色だけの場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 25 of 54

アルゴリズム：全体の流れ

黒マスを全部隠せるか判定

どちらか一方でも可能だったら…

白マスを全部隠せるか判定

色の種類数を 1 にできる！

例外として、 $s = HW$ の場合「色の種類数を 0 にできる」

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 26 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 5 | 4 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 27 of 54

小課題 5 は、画像が 4 色以下となっている

2 色の場合の解法を振り返って見よう

- 1 黒マスを全部隠せるか判定
- 2 白マスを全部隠せるか判定
- 3 黒・白を両方全部隠せるか判定 ($S = HW$ であるか)

小課題 5 | 4 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 28 of 54

小課題 5 は、画像が 4 色以下となっている

2 色の場合の解法を振り返って見よう → **これを 3 色にも応用したい！**

- 1 色 1 を全部隠せるか判定
- 2 色 2 を全部隠せるか判定
- 3 色 3 を全部隠せるか判定
- 4 色 1, 2 を全部隠せるか判定

- 5 色 1, 3 を全部隠せるか判定
- 6 色 2, 3 を全部隠せるか判定
- 7 色 1, 2, 3 を全部隠せるか判定
($S = HW$ であるか)

小課題 5 | 4 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 29 of 54

小課題 5 は、画像が 4 色以下となっている

2 色の場合の解法を振り返って見よう → **これを 3 色にも応用したい！**

- 1 色 1 を全部隠せるか判定
- 2 色 2 を全部隠せるか判定
- 3 色 3 を全部隠せるか判定
- 4 色 1, 2 を全部隠せるか判定

- 5 色 1, 3 を全部隠せるか判定
- 6 色 2, 3 を全部隠せるか判定
- 7 色 1, 2, 3 を全部隠せるか判定
($S = HW$ であるか)

For example ...

1, 2, 3, 5 が可能なとき、色の種類数を 1 にできる

小課題 5 | 4 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 30 of 54

さらに 4 色の場合にも応用したい → 15 個の場合を判定すればよい

- 1 色 1 を全部隠せるか判定
- 2 色 2 を全部隠せるか判定
- 3 色 3 を全部隠せるか判定
- 4 色 4 を全部隠せるか判定
- 5 色 1, 2 を全部隠せるか判定
- 6 色 1, 3 を全部隠せるか判定
- 7 色 1, 4 を全部隠せるか判定
- 8 色 2, 3 を全部隠せるか判定

- 9 色 2, 4 を全部隠せるか判定
- 10 色 3, 4 を全部隠せるか判定
- 11 色 1, 2, 3 を全部隠せるか判定
- 12 色 1, 2, 4 を全部隠せるか判定
- 13 色 1, 3, 4 を全部隠せるか判定
- 14 色 2, 3, 4 を全部隠せるか判定
- 15 色 1, 2, 3, 4 を全部隠せるか判定
($S = HW$ であるか)

小課題 5 | 4 色以下の場合

はじめに

1

2

3

4

5

6

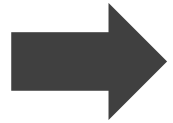
7

8

満点

Page 31 of 54

このように、「どの色を隠すか」を $2^A - 1$ 通り全部試す (A は色の種類数)



それぞれに対しては **ギリギリの長方形** を作って判定する
できる中で「残す色の種類数」が一番小さいものが答え

そうすると、計算時間 $O(2^A \times HW)$ で解くことができる

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 32 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 6 | 15 色以下の場合

はじめに

1

2

3

4

5

6

7

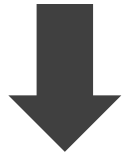
8

満点

Page 33 of 54

小課題 6 は、画像が 15 色になることまでありうる

現状の課題



改善？

「どの色を隠すか」それぞれの判定に $O(HW)$ もかけるから
全体で計算時間 $O(2^A \times HW)$ もかかっている

判定に $O(HW)$ もかけるのをやめて効率的な方法を考えよう
すると、全体の計算時間の短縮が望める

小課題 6 | 15 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

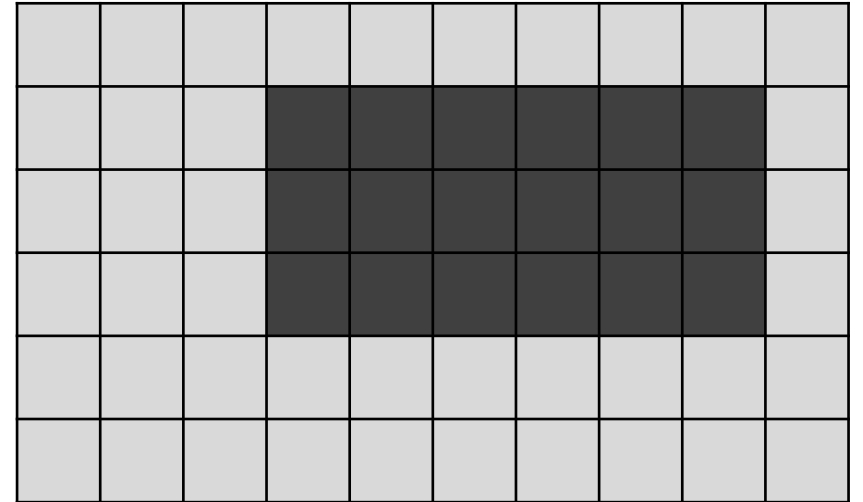
Page 34 of 54

ギリギリの長方形を求める方法を思い出してみよう

上端 : 黒マスの中で一番上の位置 ra
下端 : 黒マスの中で一番下の位置 rb
左端 : 黒マスの中で一番左の位置 ca
右端 : 黒マスの中で一番右の位置 cb

$ra \rightarrow$

$rb \rightarrow$



ca

cb

ra, rb, ca, cb をなんとかして高速に求めたい！

小課題 6 | 15 色以下の場合

はじめに

1

2

3

4

5

6

7

8

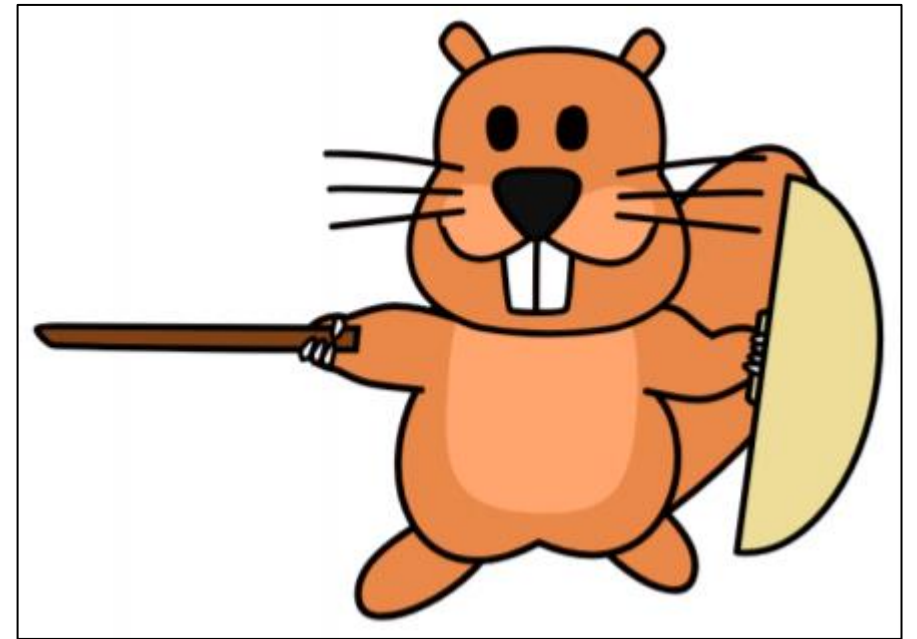
満点

Page 35 of 54

人間だとどうやって求めるか？

クイズ

右の画像で黄土色、薄茶色、濃茶色を隠すとき
 ra はどの位置になるか？



小課題 6 | 15 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 36 of 54

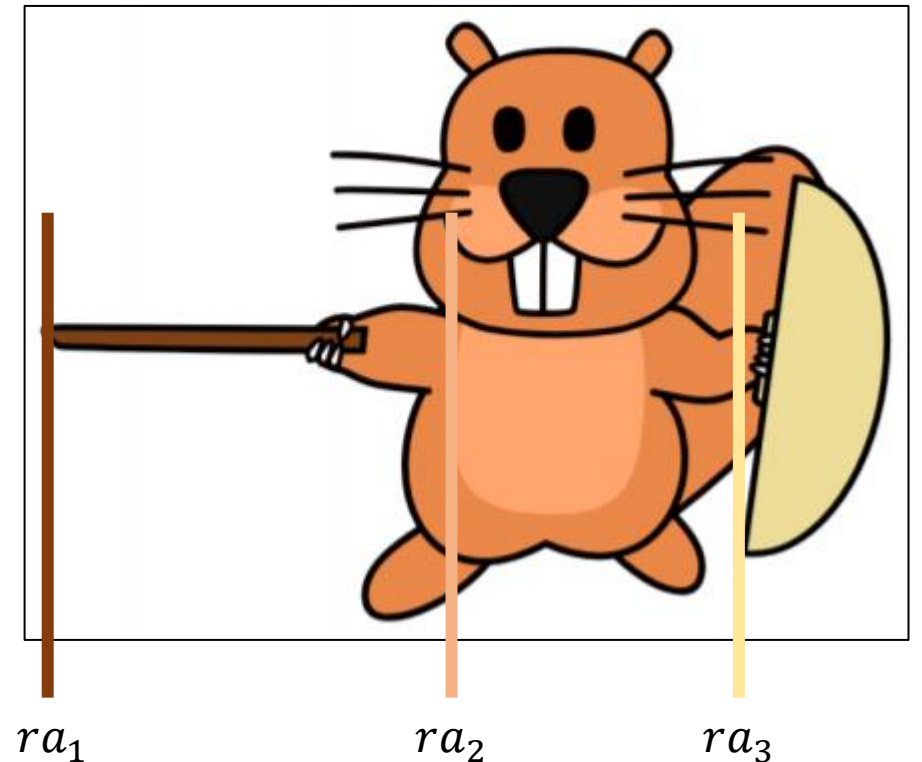
人間だとどうやって求めるか？

クイズ

右の画像で黄土色、薄茶色、濃茶色を隠すとき ra はどの位置になるか？

答え

各色に対する「最も左の位置」の中で一番左の ra_1 が答えとなる！



小課題 6 | 15 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 37 of 54

このように、各色に対して「最も左の位置」 $ra_1, ra_2, \dots, ra_{15}$ を前計算しておく

- そうすると、それぞれの隠し方に対して ra がすぐ求まる！

rb, ca, cb についても同様にやる

すると、全体で計算時間 $O(HW + 2^A \times A)$ で答えを求められる！

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 38 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 7 | 30 色以下の場合

はじめに

1

2

3

4

5

6

7

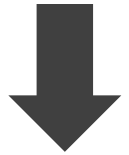
8

満点

Page 39 of 54

小課題 7 は、画像が 30 色になることまでありうる

現状の課題



改善？

「どの色を隠すか」 2^A 通りそれぞれ判定すると時間がかかる
これをなんとかしたい

実際に隠す長方形領域は $O(H^2 \times W^2)$ 通りなのでもっと少ない
同じ隠し方を無駄に探索するのをやめたい

小課題 7 | 30 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 40 of 54

ステップ 1

実は、選ばれる ra, rb, ca, cb について...

$ra : ra_1, ra_2, \dots, ra_A$ の中から選ばれる

$rb : rb_1, rb_2, \dots, rb_A$ の中から選ばれる

$ca : ca_1, ca_2, \dots, ca_A$ の中から選ばれる

$cb : cb_1, cb_2, \dots, cb_A$ の中から選ばれる

最大で A^4 通りの長方形領域しか調べなくてよい！

小課題 7 | 30 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 41 of 54

ステップ 2

ra, rb, ca, cb が決まったとき、種類数はどう判定すればいいか？

全て隠せる条件は $ra \leq ra_i, rb_i \leq rb, ca \leq ca_i, cb_i \leq cb$ だから...

- $ra \leq ra_i, rb_i \leq rb, ca \leq ca_i, cb_i \leq cb$ のとき色 i を隠せる
それ以外るとき隠せない

すると、1 つの長方形領域に対して、**残る色の種類数を計算時間 $O(A)$ で求められる！**

小課題 7 | 30 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 42 of 54

アルゴリズムの流れ

- 1 それぞれの色に対する ra_i, rb_i, ca_i, cb_i を 前計算 で求める
- 2 長方形領域 (ra, rb, ca, cb) を A^4 通り全探索する
- 3 それぞれの長方形領域に対して、残る色が何種類になるかを $O(A)$ で判定する

計算時間 $O(HW + A^5)$ で動作し、小課題 7 に正解する！

目次

はじめに

1

2

3

4

5

6

7

8

満点

Page 43 of 54

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

小課題 8 | 70 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 44 of 54

問題のおきかえ

小課題 7 では、次のような問題を解いていることになる

平面上に A 個の長方形領域がある

- i 個目の長方形領域は $ra_i \leq x \leq rb_i$ 、 $ca_i \leq y \leq cb_i$ の範囲

面積 s 以下のエリアは、最大で何個の長方形領域を包めるか？

小課題 8 | 70 色以下の場合

はじめに

1

2

3

4

5

6

7

8

満点

Page 45 of 54

面積 s ギリギリになるほどよいので...

- ra, rb, ca が決まると、 $cb = ca + S/(rb - ra)$ とセットできる

そうすることで、計算時間を $O(HW + A^4)$ に短縮できる！

目次

小課題 1	$H = 1, W \leq 10$	8 ページ
小課題 2	$H \leq 10, W \leq 10$	11 ページ
小課題 3	$S = 1$	14 ページ
小課題 4	$1 \leq A_{i,j} \leq 2$	20 ページ
小課題 5	$1 \leq A_{i,j} \leq 4$	26 ページ
小課題 6	$1 \leq A_{i,j} \leq 15$	32 ページ
小課題 7	$1 \leq A_{i,j} \leq 30$	38 ページ
小課題 8	$1 \leq A_{i,j} \leq 70$	43 ページ
小課題 9	追加の制約はない.	46 ページ

満点解法

はじめに

1

2

3

4

5

6

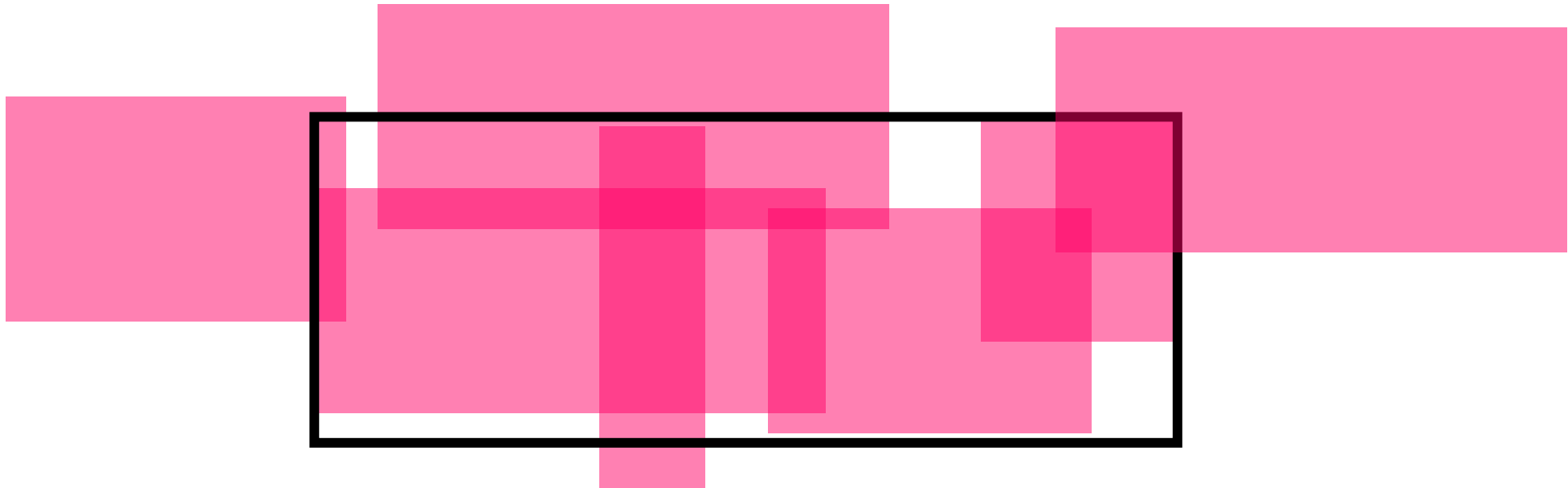
7

8

満点

Page 47 of 54

満点は、色が **最大 256 種類** ある
→ より効率的に計算する必要がある！



満点解法

はじめに

1

2

3

4

5

6

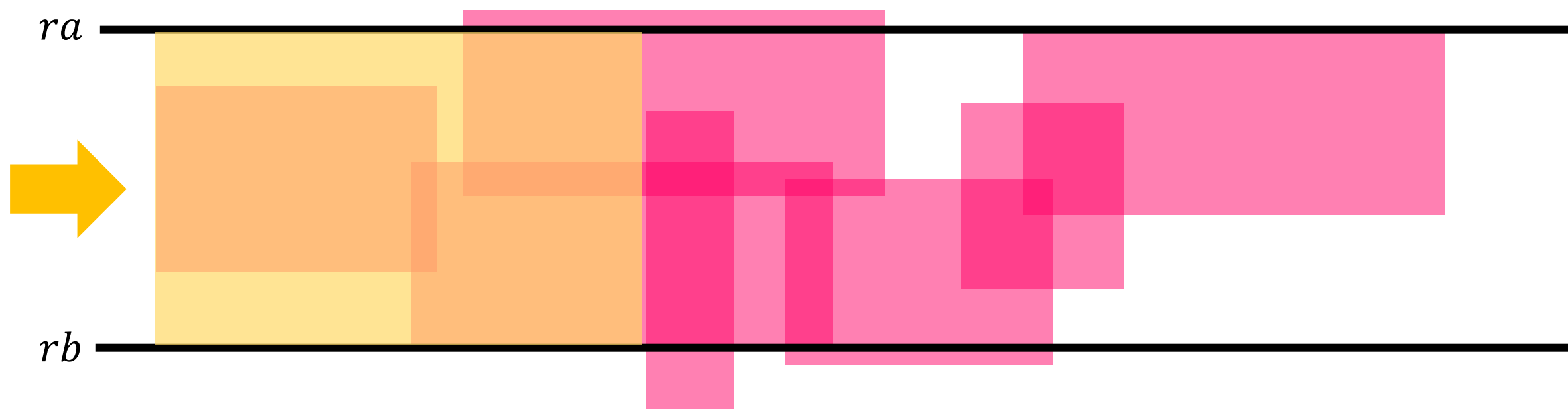
7

8

満点

Page 48 of 54

ここで、 ra と rb を固定して考えてみよう



満点解法

はじめに

1

2

3

4

5

6

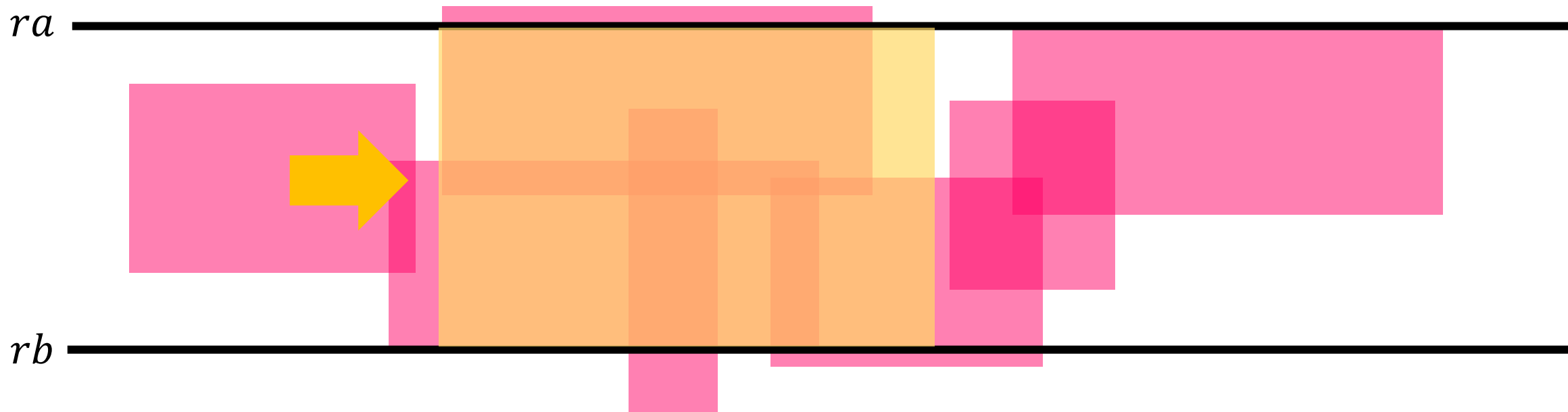
7

8

満点

Page 49 of 54

ここで、 ra と rb を固定して考えてみよう
ギリギリの長方形は、幅が決まっている！



満点解法

はじめに

1

2

3

4

5

6

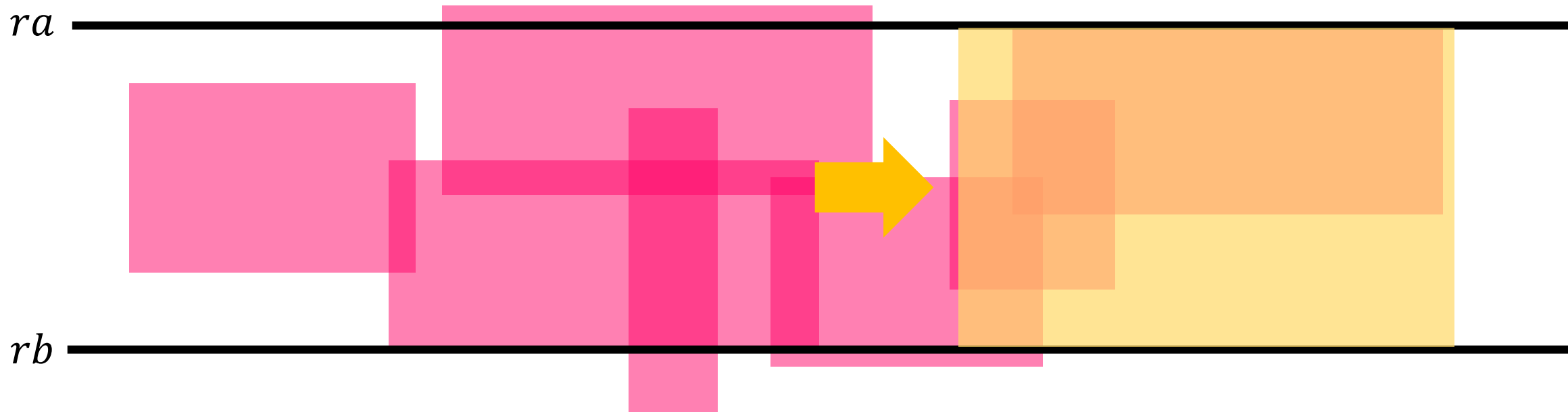
7

8

満点

Page 50 of 54

ここで、 ra と rb を固定して考えてみよう
ギリギリの長方形は、幅が決まっている！



満点解法

はじめに

1

2

3

4

5

6

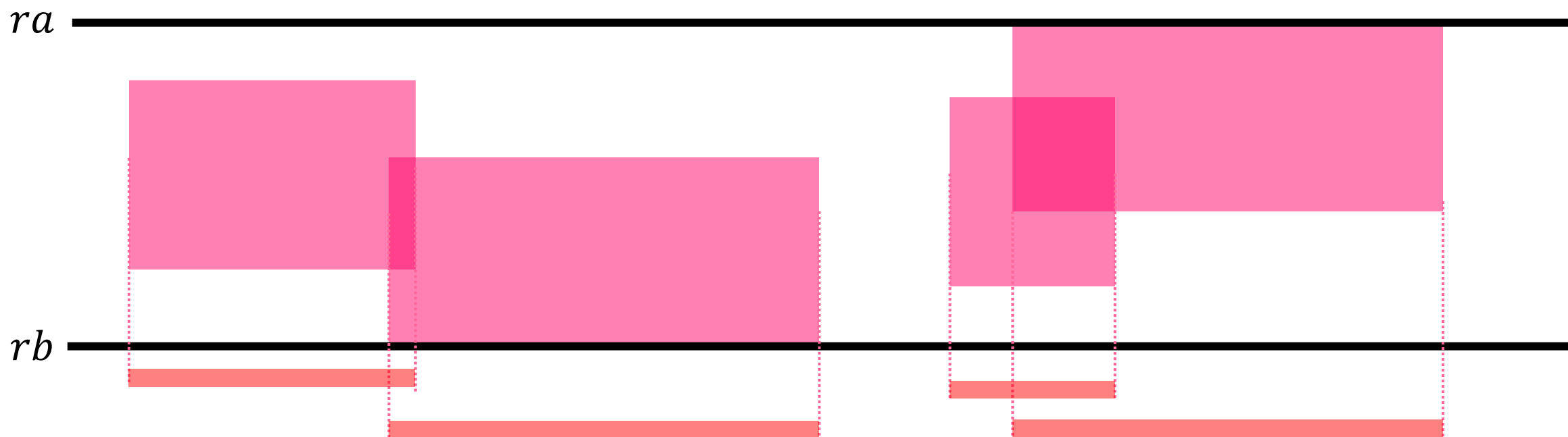
7

8

満点

Page 51 of 54

ra と rb の間に入っている長方形は、区間としてみなすことができる



満点解法

はじめに

1

2

3

4

5

6

7

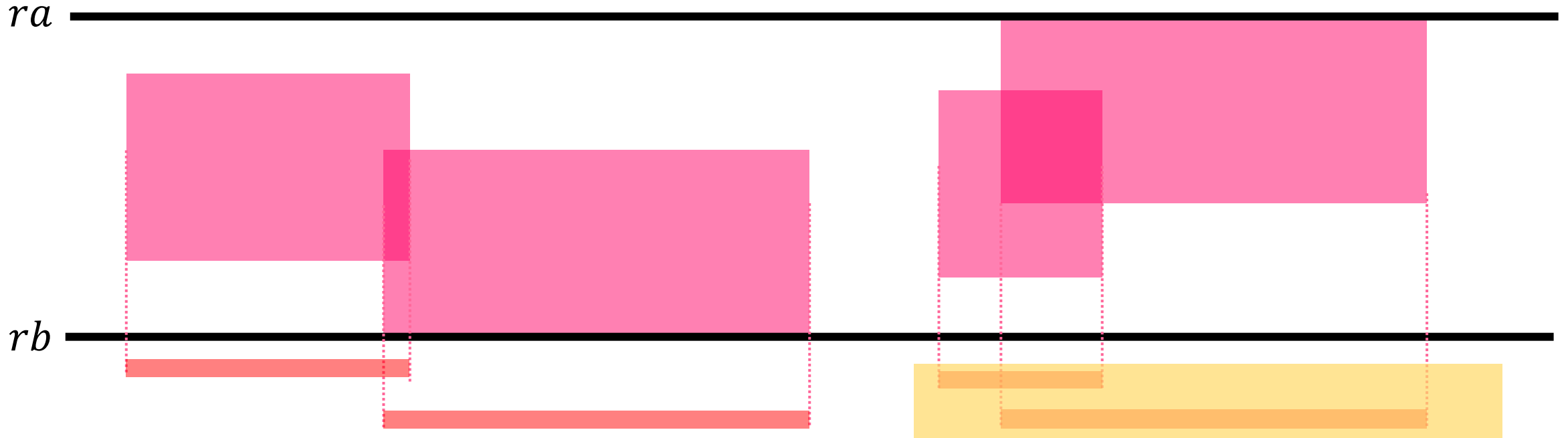
8

満点

Page 52 of 54

ra と rb の間に入っている長方形は、区間としてみなすことができる

→ 長さ $L = S/(rb - ra)$ の区間は、最大でいくつの区間を含むことができるか？



これは、尺取り法を使うことで、計算時間 $O(A)$ で求められるようになる

尺取り法のイメージ

- 区間の長さが L になるまで、区間の右端を右に移動していく
- 区間の長さが L 以上になったら、区間の左端を右に移動していく
- 見ている区間に応じて、区間が入る／出る、をシミュレートする

→ 全体計算量 $O(HW + A^3)$ で解くことができる！

得点分布

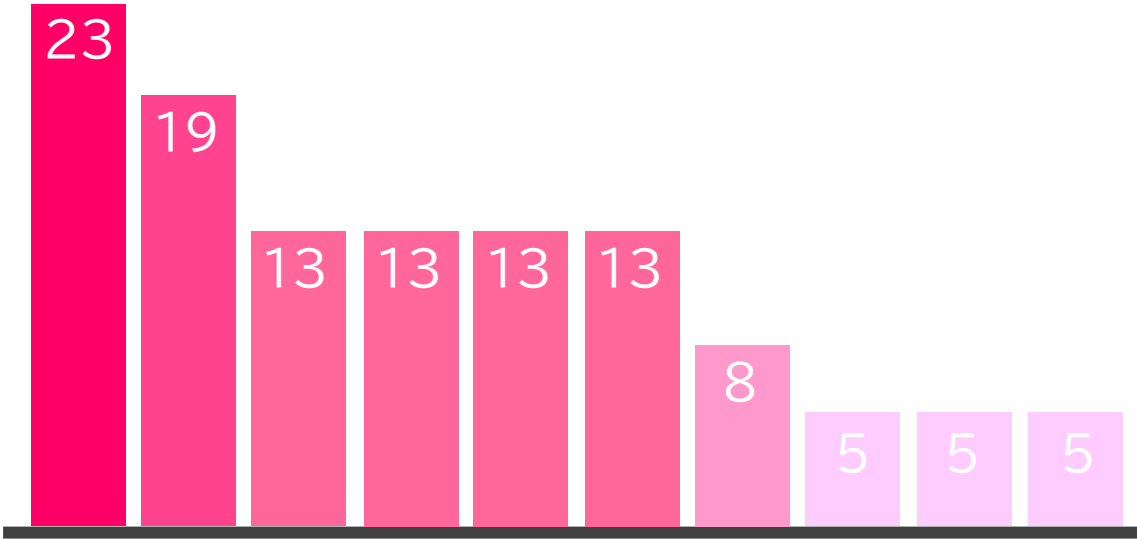
小課題別データ

#	配点	正解者
1	8 点	🏆🏆🏆🏆🏆🏆🏆
2	10 点	🏆
3	5 点	🏆🏆🏆🏆🏆🏆🏆🏆🏆
4	6 点	🏆
5	5 点	-
6	13 点	-
7	13 点	-
8	15 点	-
9	25 点	-

※ 有資格者のみを集計したデータです

得点データ

提出者 13 名（そのうち得点獲得者が 10 名）
平均点：0.9 点前後



※ 有資格者のみを集計したデータです